

Complete Agentic AI Course

LangChain · LangGraph · RAG · Vectorless RAG · Deep Agents · Guardrails · Evals · LLM Gateways · [▶ watch the video \(11h 13m\)](#)

How to use these: open one card at a time, tick "got it", and turn on  focus mode if your mind wanders.



The big picture (read this first)

LangChain v1 1

models · tools · messages · structured output

0:00 – 1:54

Middleware + LangGraph 2

hooks · summarize · HITL · state graphs

1:54 – 3:50

LangGraph agents 3

ReAct · memory · streaming · workflows

3:50 – 5:46

RAG, end to end 4

load → chunk → embed → retrieve
→ answer

5:46 – 7:36

Vectorless · Deep agents · Guardrails 5

tree index · planners · safety layers

7:36 – 9:28

Evals + Gateways 6

LangSmith judges · LiteLLM routing

9:28 – 11:13



How "agentic" grows

Plain LLM

input → model → output. Its knowledge stops at the training cutoff.



Tool agent

model decides to call one tool and uses the result as context



ReAct

loops reason → act → observe across many tools



LangGraph workflow

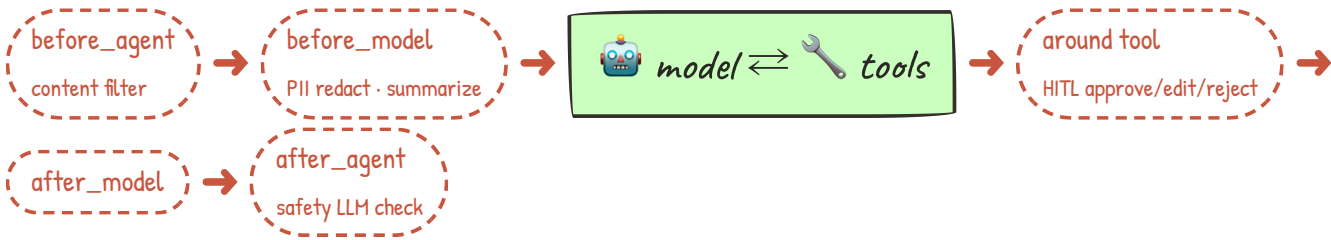
explicit state, nodes, edges, memory, human checkpoints



Deep agent

planner to-do list, sub-agents, long system prompt, shared file system

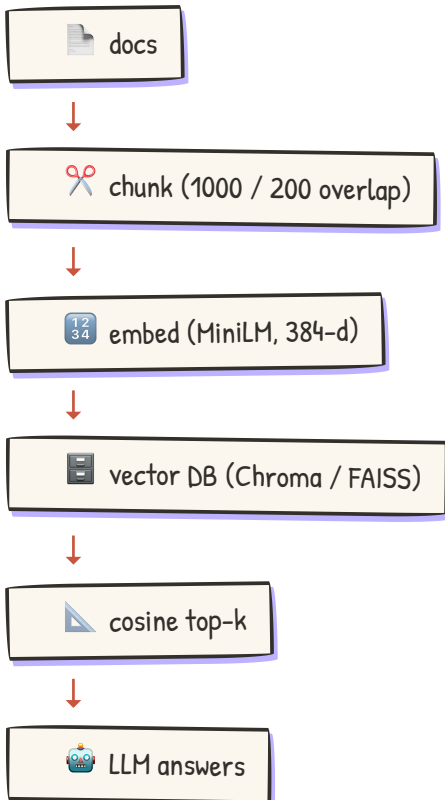
Where middleware & guardrails sit



Put the cheap rules (regex, keywords) first and the expensive LLM checks last. A request blocked in `before_agent` costs nothing.

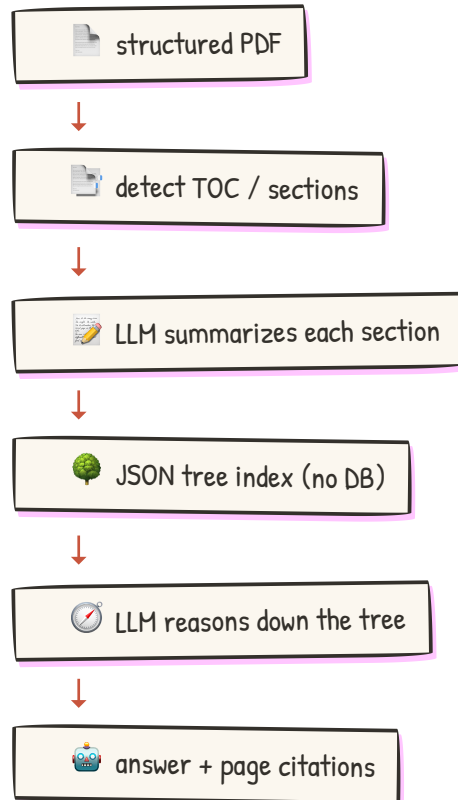
Vector RAG vs Vectorless RAG

Traditional (vector)



fast · cheap · scales to millions of docs

Vectorless (PageIndex)



reasoning · keeps sections whole · explainable

Choose based on your documents, not the hype. For mixed workloads, use a hybrid.

The 4 RAG eval metrics

Correctness

Answer relevance

Groundedness

answer vs ground truth
needs a reference answer

answer vs question

answer vs retrieved docs
catches hallucination

Retrieval relevance

docs vs question



Part 1 · 0:00 – 1:54

LangChain v1 Foundations



Course Roadmap

0:00:00–0:04:00



Big idea: The 11-hour course goes from LangChain v1 basics to agentic systems, retrieval, safety and evaluation.

Key concepts:

- ♦ **Module order** → LangChain v1, then LangGraph (agentic apps), then RAG (traditional, agentic, vectorless, and vector vs vectorless compared), then Deep Agents / deep research agents, then AI security (guardrails), then LLM evals with open-source libraries, then LLM gateways (~30–40 min).
- ♦ **LangChain v1 changes** → new agent-creation API, model integrations, tool calling, structured output, message types, short-term memory, streaming, **middleware** (built-in and custom), and guardrails.

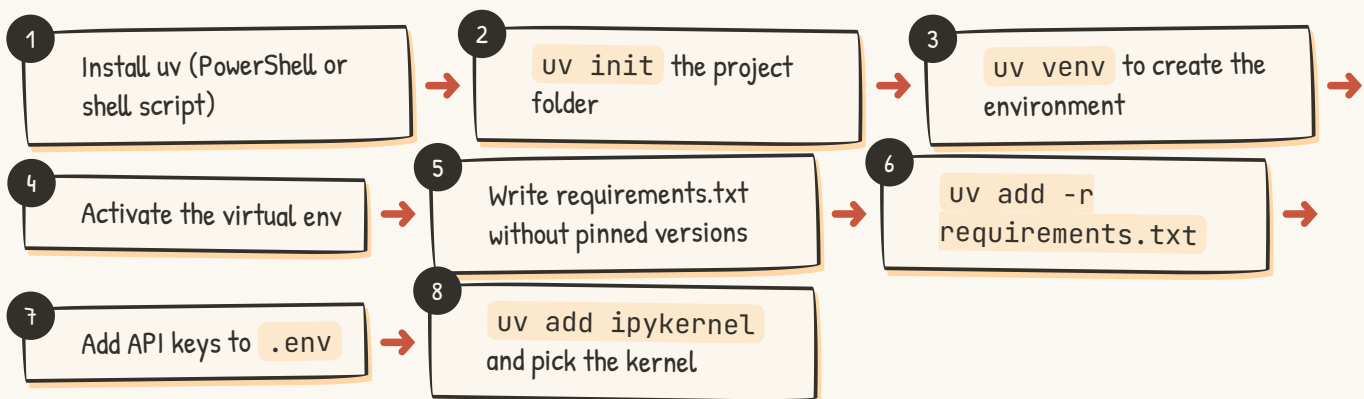
Environment Setup with uv 0:04:00-0:20:00

 **Big idea:** Use the Rust-based `uv` package and project manager for fast, reproducible environments. The IDE used is Google Antigravity, an agentic IDE similar to Cursor or VS Code.

Key concepts:

- ✦ `uv init` : turns the folder into a project and creates `pyproject.toml`, `main.py` and `.python-version` (Python 3.13 here).
- ✦ `uv venv` : creates `.venv`.
- ✦ Activate with `.venv\Scripts\activate` on Windows or `source .venv/bin/activate` on Mac/Linux.
- ✦ `uv add -r requirements.txt` : installs everything and records the resolved versions in `pyproject.toml`. This replaces `pip install -r`.
- ✦ `uv add <pkg>` : installs a single package, e.g. `ipykernel` for Jupyter.

Flow:



Code essentials: requirements are `langchain`, `langchain-community`, `langchain-openai`, `langchain-groq`, `langchain-google-genai`, `python-dotenv`. Versions installed: `langchain 1.1.0`, `langchain-google-genai 3.2.0`.

watch out

- Create `requirements.txt` outside `.venv`.
- Check `pyproject.toml` to see the exact installed versions.
- Stay on the latest LangChain, because APIs get deprecated or moved between packages.

→ **You need three keys** → OpenAI, Google AI Studio and Groq. Load them with `load_dotenv()` and `os.getenv()`.

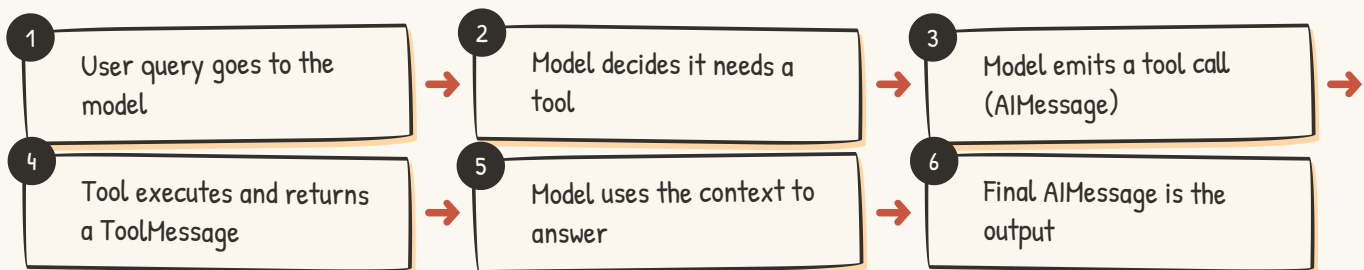
What Is an Agent? + `create_agent` 0:22:00-0:38:00

 **Big idea:** A plain LLM only knows data up to its training cutoff. An agent is an LLM that decides on its own when to call an external tool, uses the tool's result as context, and then answers.

Key concepts:

- ♦ **Gen AI app** → input goes to the LLM, which returns output. There are no tools.
- ♦ **Tool** → any external capability, such as a third-party API, web search or a database.
- ♦ **Context** → what the tool returns, which is passed back to the LLM.
- ♦ **ReAct architecture** → the older way of building agents by hand (LLM + tool + manual wiring). In v1 this is collapsed into `create_agent`.
- ♦ **Docstring** → the LLM reads a tool's docstring to decide which tool to call.

Flow:



Compare:

	Plain LLM (Gen AI app)	Basic agent
Knowledge	Frozen at training cutoff	Can reach current data through tools
Decision-making	None	Picks whether and which tool to call
Graph	start → model → end	start → model ⇌ tools → end

Code essentials:

```
from langchain.agents import create_agent
def get_weather(city: str) → str:
```

```
"""Get the weather for a city."""
return f"It's sunny in {city}"
agent = create_agent(model="gpt-5", tools=[get_weather], system_prompt="You are a hel
resp = agent.invoke({"messages": [{"role": "user", "content": "Weather in New York?"}]
print(resp["messages"][-1].content)
```

! *watch out*

- `verbose=True` is not accepted by `create_agent`.
- Input must be a dict with a `messages` key. Passing a plain string raises "expected dictionary".
- `resp["messages"]` holds the whole trace (Human, AI tool call, Tool, AI). `[-1]` is the final answer.
- Rendering the agent object shows its graph diagram.
- Misspelled user input still works.

 **Big idea:** There are two ways to load a chat model. One is the provider-agnostic `init_chat_model("provider:model")`. The other is the provider-specific chat class.

Key concepts:

- ✦ `init_chat_model("gpt-4.1")`: the OpenAI provider is inferred from the model name and you get back a `ChatOpenAI`.
- ✦ `init_chat_model("google_genai:gemin-2.5-flash-lite")`: Gemini.
- ✦ `init_chat_model("groq:qwen/qwen3-32b")`: an open model on Groq. Qwen3 is a reasoning model, so its output includes reasoning text.
- ✦ `.invoke("text")` returns an `AIMessage`. Use `.content` to get just the text.

Compare:

Provider	<code>init_chat_model</code> string	Dedicated class (package)
OpenAI	"gpt-4.1"	<code>ChatOpenAI</code> (<code>langchain_openai</code>)
Google	"google_genai:gemin-2.5-flash-lite"	<code>ChatGoogleGenerativeAI</code> (<code>langchain_google_genai</code>)
Groq	"groq:qwen/qwen3-32b"	<code>ChatGroq</code> (<code>langchain_groq</code>)

Code essentials: `from langchain.chat_models import`
`init_chat_model` (the name is singular; `init_chat_models` fails to import).
`model = init_chat_model("groq:qwen/qwen3-32b");`
`model.invoke("Why do parrots talk?").content`

watch out

- A typo in the model name (e.g. "gtp-4.1") gives "unable to infer provider".
- The first call to Gemini can be slow.



Streaming and Batch

0:50:00-0:57:00



Big idea: `invoke` waits for the whole response. `stream` returns tokens as they are generated, which is better UX for chatbots. `batch` sends independent prompts in parallel, which improves speed and cost.

Key concepts:

- ✦ `model.stream(prompt)` returns a generator of chunks. Iterate over it and print `chunk.text`.
- ✦ `model.batch([q1, q2, q3])` runs the requests in parallel and returns a list of responses.
- ✦ `config={"max_concurrency": 5}` caps the number of parallel calls. For example, 10 prompts go out as two waves of 5.

Compare:

Method	Returns	Use when
<code>invoke</code>	One full <code>AIMessage</code> after generation ends	Short or simple calls
<code>stream</code>	Iterator of chunks	Chat UIs and long outputs
<code>batch</code>	List of <code>AIMessages</code>	Many independent inputs

Code essentials:

```
for chunk in model.stream("Write 200 words on AI"):
    print(chunk.text, end="|", flush=True)
model.batch(["Why are parrots colorful?", "How do planes fly?"], config={"max_concurr
```

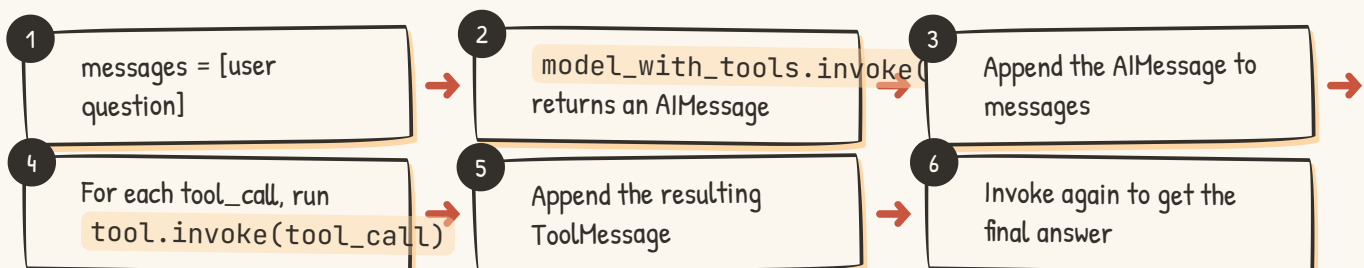
Gotchas/tips: calling `stream` alone just returns a generator object. You must loop over it.

 **Big idea:** A tool is a schema (name, args, description) paired with a function. You attach tools to a model with `bind_tools` or `create_agent`, and you can run the tool-execution loop by hand.

Key concepts:

- ◆ `@tool` decorator (`from langchain_core.tools / langchain.tools`): turns a function into a LangChain tool.
- ◆ Docstring and type hints become the tool's schema. The LLM uses the docstring to choose a tool.
- ◆ `model.bind_tools([get_weather])` : the older binding style, still valid.
`create_agent` is the newer alternative.
- ◆ `response.tool_calls` : a list of `{name, args, id}`, e.g.
`name='get_weather', args={'location':'Boston'}`.
- ◆ **Tool execution loop** → you run the tool calls yourself and feed the results back.

Flow: (manual loop):



Code essentials:

```
from langchain.tools import tool
@tool
def get_weather(location: str) → str:
    """Get weather at a location."""
    return f"It's sunny in {location}"
mwt = model.bind_tools([get_weather])
ai = mwt.invoke(msgs); msgs.append(ai)
for tc in ai.tool_calls: msgs.append(get_weather.invoke(tc))
final = mwt.invoke(msgs)
```

 watch out

- The function body can hit a real API or database. The return value becomes the context.
- Calling `tool.invoke(tool_call)` with the full `tool_call` dict returns a `ToolMessage` that carries the matching id.



Big idea: Messages are LangChain's basic unit of model context. Each one has a role, content (text, audio or documents) and optional metadata, and together they carry conversation state.

Key concepts:

- ✦ **SystemMessage** : instructions for how the model should behave and what persona to take.
- ✦ **HumanMessage** : user input. A plain string passed to `invoke` is treated as one.
- ✦ **AIMessage** : model output (text, `tool_calls`, metadata). You can also write one by hand to inject history.
- ✦ **ToolMessage** : a tool's result, linked to the call by `tool_call_id`.
- ✦ **Text prompt** → a single string. Use it for one-off requests with no history and minimal code.
- ✦ **Message prompt** → a list of message objects. Use it for multi-turn history, personas and few-shot examples.
- ✦ **Metadata** → optional fields such as `name` (to tell users apart) and `id` (for tracing).
- ✦ `response.usage_metadata` : input, output and total tokens.

Compare

Text prompt	Message prompt
<code>model.invoke("What is LangChain?")</code>	<code>model.invoke([SystemMessage(...), HumanMessage(...)])</code>
No history, no persona	History, roles and system instructions

Code essentials:

```
from langchain.messages import SystemMessage, HumanMessage, AIMessage, ToolMessage
msgs = [SystemMessage("You are a senior Python dev; give code + reasoning"),
        HumanMessage("Can you help me?"), AIMessage("Happy to help."),
        HumanMessage("How do I create a REST API?")]
model.invoke(msgs)
# Tool case: AIMessage(content="", tool_calls=[{"name": "get_weather", "args": {"location": "London"}},
# ToolMessage(content="Sunny, 72F", tool_call_id="call_1")
```

watch out

- A detailed system prompt gives much more specific answers. "Helpful coding assistant" produced a generic reply. "Senior Python dev, give code, be concise" produced a Flask example with production tips.
- A hand-written AIMessage plus ToolMessage pair lets you simulate a tool round-trip without running the tool.



Big idea: Force the LLM to return data that matches a schema, so later code can consume it. Pydantic is the richest option, with runtime field validation, descriptions and nesting.

Key concepts:

- ◆ **BaseModel** + **Field(description=...)** : the field descriptions tell the LLM where each value goes.
- ◆ **model.with_structured_output(Movie)** returns a **RunnableBinding** (model plus Pydantic tool parser). **invoke** returns a **Movie** instance.
- ◆ **include_raw=True** : returns both the raw **AIMessage** and the parsed object.
- ◆ **Nested structure** → a field can be a **list[Actor]** or **list[str]**, and fields can be optional (**budget: float | None = None**).
- ◆ **Validation** → a wrong type, such as an **int** where a **str** is expected, raises an error.

Code essentials:

```
from pydantic import BaseModel, Field
class Actor(BaseModel): name: str; role: str
class MovieDetails(BaseModel):
    title: str = Field(description="Movie title"); year: int
    cast: list[Actor]; genres: list[str]
    budget: float | None = Field(None, description="Budget in million USD")
model.with_structured_output(MovieDetails).invoke("Details of Inception")
```



watch out

- Plain **invoke** gives free text. The structured model gave **title=Inception**, **year=2010**, **director=Nolan**, **rating=8.8**, plus cast with roles and **budget=160.0**.



Structured Output: TypedDict, Model Profile, Dataclasses

1:39:00–1:50:00



Big idea: TypedDict and dataclasses are lighter schema options with no runtime validation. Any of the three schema types can also be passed to `create_agent(response_format=...)`.

Key concepts:

- ◆ **TypedDict** + `Annotated[type, "description"]` (from `typing_extensions`): behaves as a plain dict at runtime and does not enforce types.
- ◆ **model.profile**: describes what the model can do, e.g. max input/output tokens, image/audio/video input, reasoning output, tool calling. For Qwen3 it shows no multimodal input, plus reasoning and tool calling.
- ◆ **@dataclass** (Python 3.7+): a class that mainly holds data, with no validation.
- ◆ `create_agent(model="gpt-5", tools=[], response_format=ContactInfo)`: read the parsed result from `result["structured_response"]`.

Compare:

Schema type	Runtime validation	Nesting	Output type
Pydantic BaseModel	Yes	Yes	Model instance
TypedDict	No	Yes	dict
@dataclass	No	Yes	dataclass instance

Code essentials:

```
from typing_extensions import TypedDict, Annotated
class MovieDict(TypedDict):
    title: Annotated[str, ..., "Movie title"]; year: Annotated[int, ..., "Release year"]
agent = create_agent(model="gpt-5", response_format=ContactInfo)
agent.invoke({"messages":[{"role":"user","content":"Extract contact: John Doe, john@e"}]})
```



watch out

- `.profile` only works on the base model. On a `with_structured_output` runnable it raises "RunnableSequence has no attribute profile".
- Choose Pydantic when you need guarantees, and TypedDict or dataclass when you want minimal overhead.

★ *Part summary* —

- ◆ LangChain v1 (1.1.0) simplifies agent building: `create_agent(model, tools, system_prompt, response_format)` replaces hand-wired ReAct agents and shows a start → model ⇌ tools → end graph.
- ◆ Models load through `init_chat_model("provider:model")` or through provider classes (`ChatOpenAI`, `ChatGoogleGenerativeAI`, `ChatGroq`). You execute them with `invoke`, `stream` (token chunks) or `batch` (parallel, with `max_concurrency`).
- ◆ Tools are `@tool`-decorated functions whose docstring and type hints form the schema. You attach them with `bind_tools` or `create_agent`, and you can drive the AI tool_call → ToolMessage → AI loop by hand.
- ◆ Messages (System, Human, AI, Tool, each with role, content and metadata) carry conversation state. Detailed system prompts noticeably improve answers, and `usage_metadata` shows token counts.
- ◆ Structured output uses `with_structured_output(schema)` or `response_format`. Pydantic validates and supports nesting and `include_raw`. TypedDict and dataclasses are lightweight with no validation. `model.profile` lists what the model supports.



Part 2 · 1:54 – 3:50

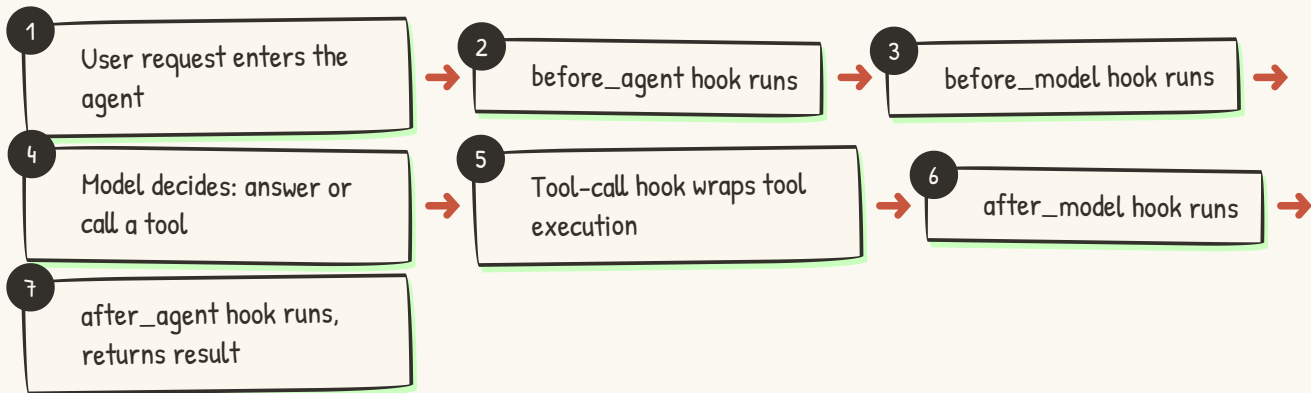
Middleware + LangGraph Basics

 **Big idea:** Middleware lets you tightly control what happens inside an agent. You attach logic at fixed "hook" points around the model call and tool calls, much like an airport has security, immigration and a boarding-pass check in sequence before you reach the gate.

Key concepts

- ◆ **Middleware:** a pluggable layer that runs before or after an agent step. Uses include logging, analytics, debugging, prompt transformation, tool selection, error handling and summarization.
- ◆ **Hooks:** the trigger points where middleware runs. These are before agent, before model, around tool calls, after model, and after agent.
- ◆ **Base agent:** a ReAct loop. The model decides whether it needs a tool, the tool runs and returns context, and the model produces the answer.
- ◆ **Stacking:** you can chain several middlewares (MW1 → MW2 → MW3) on one agent.

Flow (agent with middleware)



Built-in middlewares mentioned

- ◆ **SummarizationMiddleware** → compresses old history as it nears the token limit
- ◆ **HumanInTheLoopMiddleware** → pauses so a human can approve tool calls
- ◆ **ModelCallLimit** → caps the number of model calls (controls cost, prevents infinite loops; has thread and run limits)
- ◆ **ToolCallLimit** → caps the number of tool executions
- ◆ **ModelFallback** → switches to a backup model if the primary fails (for example, a bad API key)
- ◆ To-do list, LLM tool selector (filters irrelevant tools per query to save tokens), tool retry



Big idea: Once the conversation grows past a threshold, older messages are automatically summarized by an LLM and only the recent context is kept. This keeps long, multi-turn chats inside the token budget.

Key concepts

- ♦ **trigger:** when to summarize. It can be based on the number of messages, the number of tokens, or a fraction of the model's context window.
- ♦ **keep:** how much recent context to keep unsummarized (a number of messages or tokens).
- ♦ **Summarizer model:** use a cheap model, because summarization runs again every time the threshold is crossed.
- ♦ **Checkpoint** (`InMemorySaver`): stores conversation state per thread. The `thread_id` in the config identifies the user or session.
- ♦ **Good fits:** long-running chatbots and multi-turn dialogues where the history matters.

Compare: trigger types

Trigger	Demo setting	Behavior observed
Messages	<code>("messages", 10)</code> , keep last 4	Message count went 2, 4 ... 10, then dropped to 6 with a "summary of conversation" message
Tokens	<code>("tokens", 550)</code> , keep 200 tokens	Token count went 149 → 302 → 456, then dropped to 396 and 232 after summaries
Fraction	<code>("fraction", 0.005)</code> (0.5% of the context window)	Summarizes once usage passes that share of the model's context window

Code essentials

- ♦ `from langchain.agents import create_agent`
- ♦ `from langchain.agents.middleware import SummarizationMiddleware`
- ♦ `from langchain.checkpoint.memory import InMemorySaver` (spoken in the video; the current import path is `langgraph.checkpoint.memory`)

```
✦ from langchain_core.messages import HumanMessage,  
SystemMessage
```

```
agent = create_agent(model="gpt-4o-mini", tools=[search_hotels],  
                    checkpointer=InMemorySaver(),  
                    middleware=[SummarizationMiddleware(model="gpt-4o-mini",  
                                                        trigger=("tokens", 550), keep=("tokens", 200))])  
config = {"configurable": {"thread_id": "test1"}}  
resp = agent.invoke({"messages": [HumanMessage(q)]}, config)  
len(resp["messages"])
```

⚠ *watch out*

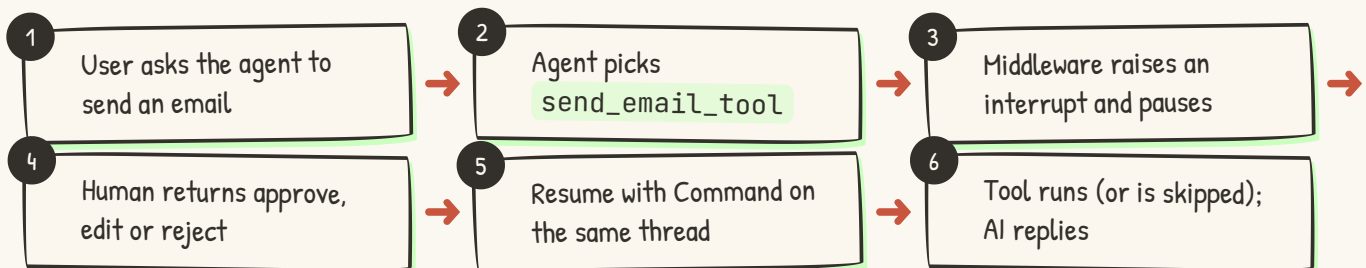
- The demo estimated tokens with a rough rule of 4 characters $\approx$ 1 token.
- Model-name typos ("gtp-4o-mini") produce an "unable to infer model" error.
- Real chatbots use much larger thresholds than 10 messages. The demo used small ones so the effect is visible.
- To add more middlewares, append them to the `middleware=[...]` list.

 **Big idea:** For high-stakes tool calls, the agent pauses before running the tool and waits for a human to approve, edit or reject it. Autonomous agents make mistakes, so critical actions need a human check.

Key concepts

- ◆ **Use cases:** database writes, financial transactions such as buying stocks, compliance workflows that require oversight, and long conversations guided by human feedback.
- ◆ **interrupt_on:** maps each tool name to its interrupt policy. A tool set to `False` runs without asking.
- ◆ **allowed_decisions:** `approve`, `edit`, `reject`.
- ◆ **Interrupt:** the result dict contains `__interrupt__` when execution is paused.
- ◆ **Command(resume=...):** resumes the paused run on the same thread and carries the human's decision.

Flow



Compare: decisions

Decision	Resume payload	Outcome
approve	<code>{"type": "approve"}</code>	Tool runs as proposed ("Email sent to john@test.com")
reject	<code>{"type": "reject"}</code>	Tool is skipped; result says the user rejected the tool call
edit	<code>{"type": "edit", "edited_action": {"name": "send_email_tool", "args": {...}}}</code>	Tool runs with the corrected arguments (wrong recipient replaced)

Code essentials

- ✦ `from langchain.agents.middleware import HumanInTheLoopMiddleware`
- ✦ `from langgraph.types import Command`

```
agent = create_agent(model="gpt-4o", tools=[read_email_tool, send_email_tool],
                    checkpoint=InMemorySaver(),
                    middleware=[HumanInTheLoopMiddleware(interrupt_on={
                        "send_email_tool": {"allowed_decisions": ["approve", "edit", "reject"]},
                        "read_email_tool": False})])
r = agent.invoke({"messages": [HumanMessage("Send email to ...")]}, config)
if "__interrupt__" in r:
    r = agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}), config)
```

⚠ watch out

- **Command** comes from `langgraph.types`, not from `langchain`. The demo first hit a `NameError` because the import was missing.
- Resume with the same `thread_id` config. The checkpoint is what lets the paused run continue.
- The email tools were mocks. A real implementation would need SMTP.

📝 LangGraph crash course roadmap 2:34:03–2:38:01



Big idea: The course is split into three parts of about 2–3 hours each, going from fundamentals to multi-agent systems to production.

Key concepts

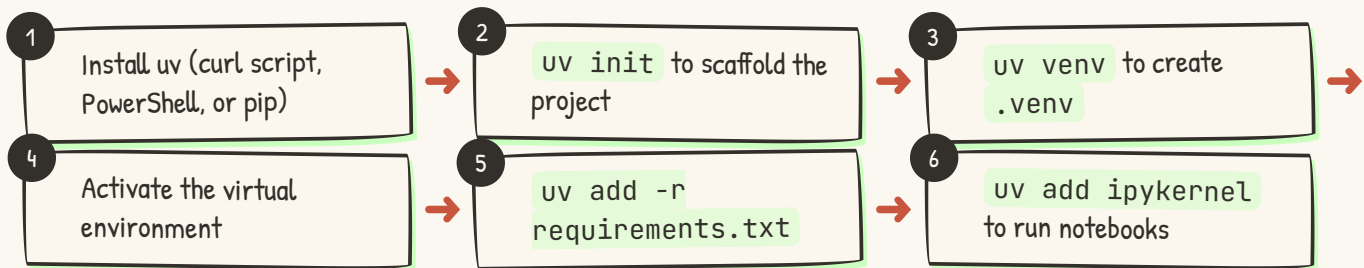
- ✦ **Part 1, fundamentals:** chatbot, tools, multiple tools, memory, human-in-the-loop, streaming, MCP built from scratch, and state, graphs, nodes and edges using the Graph API.
- ✦ **Part 2, advanced:** workflows and agents, agents communicating with each other on complex workflows, multi-agent state management, the Functional API, and debugging in LangGraph Studio with LangSmith.
- ✦ **Part 3, production:** end-to-end projects, LLM Ops pipelines, deployment to Hugging Face Spaces, LLM evaluation metrics with MLflow, AWS and Grafana.

💡 *Big idea:* Use `uv`, a very fast Python package and project manager written in Rust, to set up the project instead of conda or pip.

Key concepts

- ◆ `uv`: 10-100× faster than pip. One tool replaces pip, pip-tools, pipx, poetry, pyenv, twine and virtualenv. It provides a universal lock file and manages Python versions.
- ◆ `uv init`: creates `.gitignore`, `.python-version` (3.13), `main.py` and `pyproject.toml`.
- ◆ `requirements.txt`: `langgraph`, `langchain`, `langsmith` (tracing and evaluation); later `python-dotenv`, `langchain-groq`, `langchain-tavily`.
- ◆ `LangSmith`: tracks and evaluates applications.

Flow



⚠️ watch out

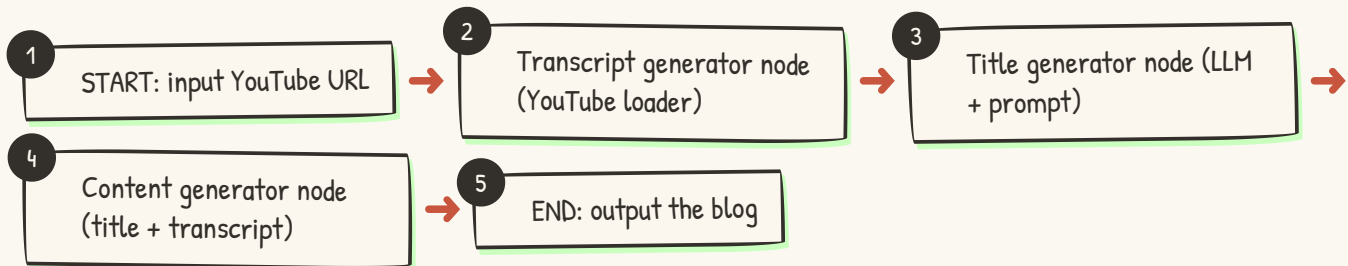
- Notebooks need `ipykernel` installed, and you select the venv as the kernel.
- After editing `.env`, restart the kernel so `load_dotenv()` picks up the new keys. The demo got an "invalid API key" error until it restarted.

💡 *Big idea:* A LangGraph workflow is a 💡 *StateGraph*. Nodes do the work, edges define the order, and a shared state object holds the variables that every node can read and write.

Key concepts

- ✦ **Node:** a unit of work backed by a Python function (the node implementation), often an LLM plus a prompt.
- ✦ **Edge:** the path information takes from one node to the next.
- ✦ **State:** a shared schema of variables (for example `transcript`, `title`, `content`) that any node in the graph can access.
- ✦ **StateGraph:** called that because it keeps state across every node.
- ✦ **START / END:** special entry and exit nodes.
- ✦ **Graph API vs Functional API:** two ways to build. The speaker recommends the Graph API as the easier way to learn.
- ✦ **State vs memory:** state is not the same as external memory. Memory is covered later.

Flow (YouTube-to-blog example)



Compare: Graph API vs Functional API

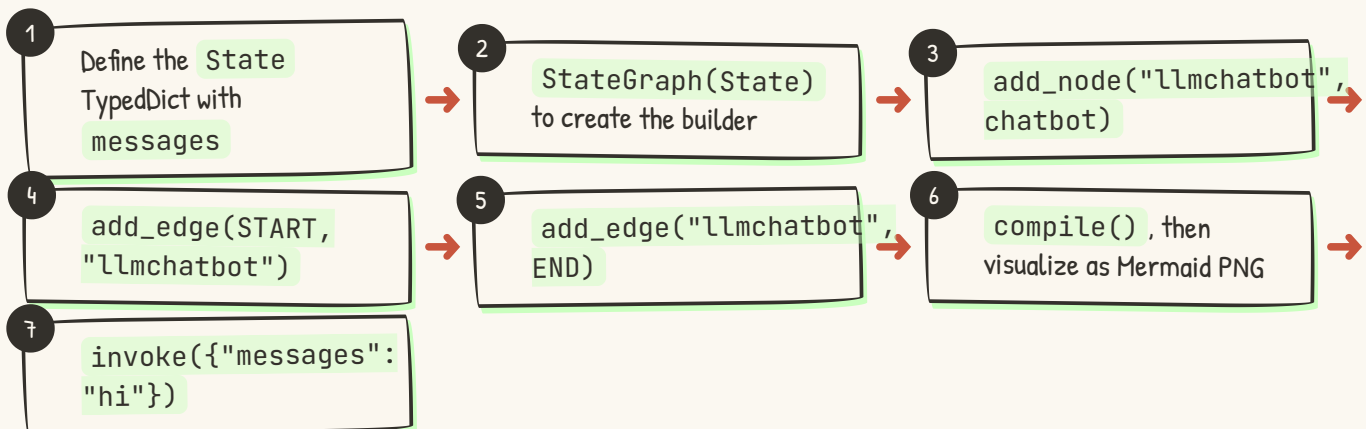
Graph API	Functional API
Explicit nodes, edges, state, and a visual graph	Write the workflow directly as functions
Recommended for learning	For users with more experience

Big idea: The smallest LangGraph app is `START` → chatbot node (an LLM) → `END`, with a `messages` state field that keeps the conversation by appending to it rather than replacing it.

Key concepts

- ◆ **TypedDict state:** at runtime the state is just a plain dict with typed keys.
- ◆ **Annotated:** attaches metadata (here, the reducer) to a type. Other code can ignore the metadata.
- ◆ **Reducer (`add_messages`):** defines how a state key gets updated. It appends new messages to the list instead of overwriting them.
- ◆ **Auto-typing:** a plain string input becomes a `HumanMessage`, and the LLM's reply is stored as an `AIMessage`.
- ◆ **LLM init:** `ChatGroq(model=...)` or the generic `init_chat_model("groq:llama3-8b-8192")`, which also works with `"openai:..."`.
- ◆ **Compile:** required before the graph can run.
- ◆ **invoke vs stream:** `invoke` returns the final state. `stream` yields an event per node, keyed by node name.

Flow



Code essentials

- ◆ `from typing import Annotated, from typing_extensions import TypedDict`
- ◆ `from langgraph.graph import StateGraph, START, END`

✦ `from langgraph.graph.message import add_messages`

```
class State(TypedDict):
    messages: Annotated[list, add_messages]
def chatbot(state: State):
    return {"messages": [llm.invoke(state["messages"])]}
builder = StateGraph(State); builder.add_node("llmchatbot", chatbot)
builder.add_edge(START, "llmchatbot"); builder.add_edge("llmchatbot", END)
graph = builder.compile()
graph.invoke({"messages": "hi"})["messages"][-1].content
```

✦ **Visualize** → `display(Image(graph.get_graph().draw_mermaid_png()))`
(from `IPython.display`)

✦ **Stream** → `for event in graph.stream({"messages": "Hi how are you"}): for v in event.values(): print(v["messages"][-1].content)`

⚠ *watch out*

- The input must be a dict keyed by the state field (`{"messages": ...}`). Passing a bare string fails because the node reads `state["messages"]` .
- **add_edge** takes node **names**, not functions. **add_node** takes a name and a function.
- Get the latest reply with `response["messages"][-1].content` .
- Calling **add_node** twice with the same name gives a "node already present" error.



Chatbot with tools: concept

3:24:00-3:29:00



Big idea: An LLM can't answer questions that need live data, such as recent AI news.

Binding tools to the LLM lets it decide to call a tool, which a separate tool node then runs.

Key concepts

- ◆ **Tool binding:** tells the LLM which tools exist ("its weapons"). It does **not** run the tools.
- ◆ **Docstring:** how the LLM learns what a tool does and what arguments it takes. Custom tools must have one.
- ◆ **Tool node:** the graph node that actually runs tool calls. It can hold several tools (Tavily plus custom functions).
- ◆ **Tavily:** a web search API built for LLM and RAG applications, with a free tier.



Chatbot with tools: implementation

3:29:00~3:46:00, transcript ends mid-demo

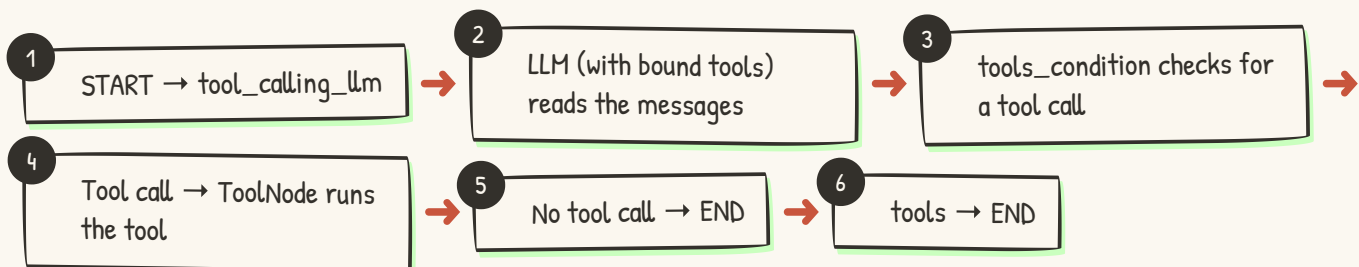


Big idea: You build `START → tool_calling_llm`, then a conditional edge that goes to `tools` if the LLM requested a tool call or to `END` otherwise, then `tools → END`.

Key concepts

- ✦ `TavilySearch(max_results=2)`: a web search tool. Invoking it returns titles, URLs and content.
- ✦ **Custom tool**: a plain typed function with a docstring (for example `multiply(a: int, b: int) → int`).
- ✦ `llm.bind_tools(tools)`: returns a `RunnableBinding` that lists the bound tools.
- ✦ `ToolNode(tools)`: a prebuilt node that runs the requested tools.
- ✦ `tools_condition`: a prebuilt router. If the latest assistant message is a tool call it routes to the node named `"tools"`, otherwise to `END`.
- ✦ **Conditional edges**: required whenever a node has more than one outgoing path.

Flow



Code essentials

- ✦ `from langchain_tavily import TavilySearch`
- ✦ `from langgraph.prebuilt import ToolNode, tools_condition`

```
tools = [TavilySearch(max_results=2), multiply]
llm_with_tools = llm.bind_tools(tools)
def tool_calling_llm(state: State):
    return {"messages": [llm_with_tools.invoke(state["messages"])]}
builder.add_node("tool_calling_llm", tool_calling_llm); builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "tool_calling_llm")
builder.add_conditional_edges("tool_calling_llm", tools_condition)
builder.add_edge("tools", END)
```

⚠️ watch out

- The tool node must be named exactly `"tools"`, because `tools_condition` routes to that name by default.
- When the LLM requests a tool, its `AIMessage` has empty `content` and a populated `tool_calls` (id, name, args). The tool output is then added as a `ToolMessage`.
- In this version the graph ends right after the tool runs (tools → END), so the LLM never writes a final answer from the tool output. ReAct agents, which loop back to the LLM, are previewed as the next step.
- Add `TAVILY_API_KEY` to `.env` and restart the kernel. Errors like "State is not defined" or "Image is not defined" mean cells need re-running after a restart.

✏️ ★ Part summary —

- ◆ LangChain middleware adds hooks before and after the agent, the model and tool calls. Built-ins include summarization, human-in-the-loop, model and tool call limits, model fallback, an LLM tool selector, and tool retry.
- ◆ **SummarizationMiddleware** compresses old history once a trigger is hit (message count, token count, or a fraction of the context window), keeps recent context, and should use a cheap model. It runs with a checkpoint and a `thread_id`.
- ◆ **HumanInTheLoopMiddleware(interrupt_on={tool: {"allowed_decisions": [...]}})** pauses on sensitive tools. You resume with `Command(resume={"decisions":[{"type": "approve" | "reject" | "edit", ...}]})` from `langgraph.types` on the same thread.
- ◆ **LangGraph basics** → a `StateGraph` made of nodes (functions), edges (order) and shared state (a `TypedDict`). `Annotated[list, add_messages]` is a reducer that appends messages. You must compile before calling `invoke` or `stream`.
- ◆ For tool use, bind tools to the LLM (docstrings tell it what each tool does), run them in a `ToolNode` named `"tools"`, and route with `add_conditional_edges(..., tools_condition)`. A ReAct-style loop back to the LLM comes next.



Part 3 · 3:50 – 5:46

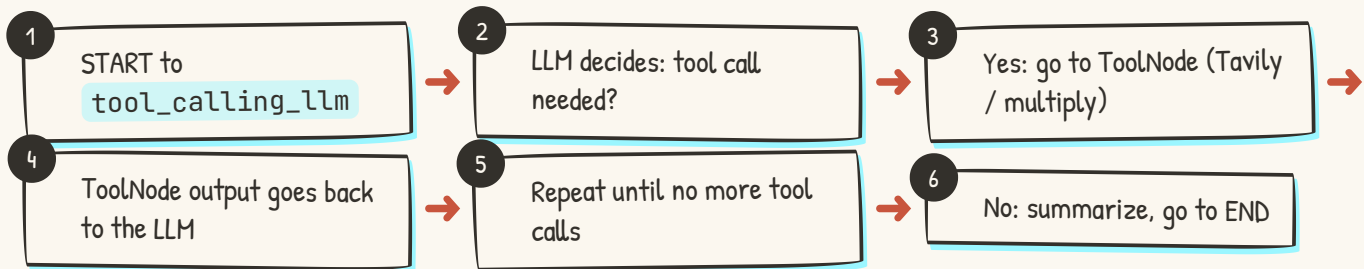
LangGraph Agents in Depth

💡 **Big idea:** In a graph where the tool node runs and then goes straight to END, a compound request ("give me AI news, then multiply 5 by 10") only gets its first part answered. If the tool output goes back to the LLM instead, the LLM can keep calling tools until every part is done. That loop is the ReAct pattern.

Key concepts:

- ♦ **Tool binding** → the LLM has the tool schemas through `bind_tools`, so it emits structured tool calls with the tool name and arguments, e.g. `multiply(5, 2)`.
- ♦ **ToolMessage** → the result of a tool, which gets added to the state messages.
- ♦ ReAct = **Act** (call a tool), **Observe** (read the tool output), **Reason** (decide whether to call another tool or finish).
- ♦ **LLM as "brain"** → it is the sole decision maker for which tool to call next and when to stop.

Flow (ReAct graph):



Compare:

Graph shape	Edge after tools	Result on a multi-part query
Simple tool-calling	tools → END	Only the first sub-task is answered
ReAct agent	tools → LLM (loop)	All sub-tasks answered and combined

Code essentials: `StateGraph`, `ToolNode`, `tools_condition`, `llm.bind_tools(tools)`. The key change:

```
builder.add_conditional_edges("tool_calling_llm", tools_condition)
```

```
builder.add_edge("tools", "tool_calling_llm") # was: ("tools", END)
```

! *watch out*

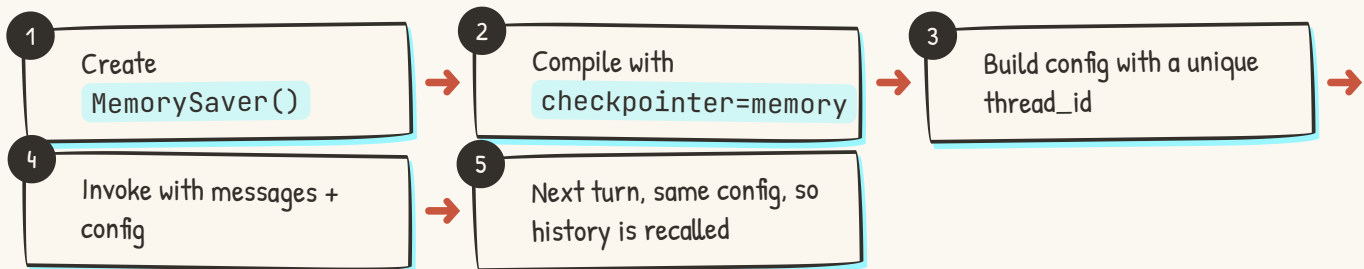
- A tool function with no `return` (the bug in the demo) gives a null ToolMessage. Always return the value.
- After you edit a tool or node you have to **recompile the graph**.
- You can add as many tools as you like. The LLM picks among them based on their docstrings.

 **Big idea:** A compiled graph forgets everything between invocations. ("My name is Krish" followed by "what is my name?" fails.) You fix this by attaching a checkpointer at compile time and passing a thread ID with each call.

Key concepts:

- ✦ **MemorySaver**: an in-memory checkpointer (backed by a defaultdict) that saves state after each node runs.
- ✦ **thread_id**: a unique session key. All turns with the same thread share history.
- ✦ **config**: `{"configurable": {"thread_id": ...}}`, passed on every `invoke` or `stream` call.

Flow:



Code essentials:

```
from langgraph.checkpoint.memory import MemorySaver
memory = MemorySaver()
graph = builder.compile(checkpointer=memory)
config = {"configurable": {"thread_id": "1"}}
graph.invoke({"messages": "Hi, my name is Krish"}, config=config)
graph.invoke({"messages": "What is my name?"}, config=config)["messages"][-1].content
```

watch out

- In a real application, keep the `thread_id` per user session.
- Calling twice on the same thread adds the messages twice. Use a new thread for a clean start.

 **Big idea:** `stream` / `astream` show you graph output as it is produced. The `stream_mode` setting controls whether you see only what each step changed or the whole accumulated state.

Key concepts:

- ◆ `.stream()` is synchronous. `.astream()` is the async version.
- ◆ `stream_mode="updates"` : emits only what the node just produced, e.g. the latest `AIMessage`.
- ◆ `stream_mode="values"` : emits the **full state** after each step, so all human and AI messages so far, growing over time.
- ◆ `astream_events` : a detailed per-event stream for debugging (async, needs a `version` argument).

Compare:

Mode	What you see per step	Use for
<code>updates</code>	Only the change from the current node	Lightweight chat output
<code>values</code>	The whole accumulated message list	Inspecting the full conversation or state
<code>astream_events</code>	Fine-grained events	Debugging

Code essentials:

```
for chunk in graph.stream({"messages": "Hi, I like cricket"}, config, stream_mode="updates"):
    print(chunk)
async for ev in graph.astream_events(inp, config, version="v2"):
    print(ev)
```

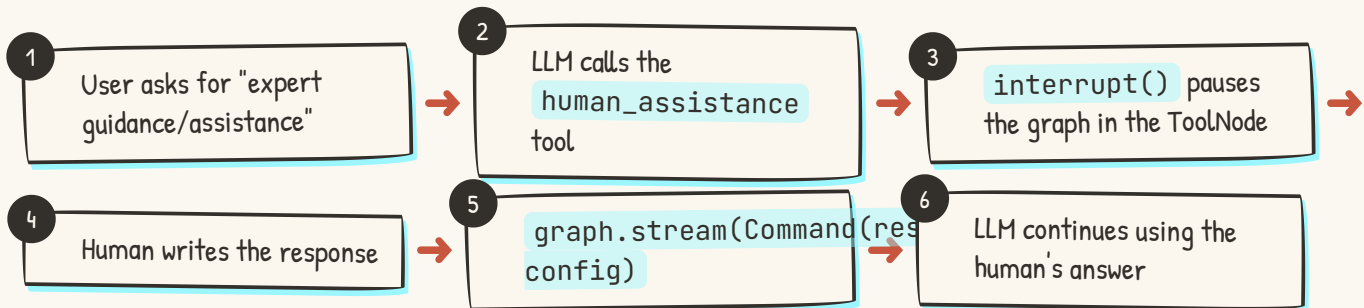
Gotchas/tips: The demo graph is just `START` → `superbot` → `END` with a `MemorySaver`.
Use a new `thread_id` when you want to compare modes from a clean history.

 **Big idea:** A graph can stop on purpose partway through, wait for a human to give feedback or approval, and then continue from the same point. This matters for workflows that should not finish without sign-off.

Key concepts:

- ✦ `interrupt(payload)` : pauses execution inside a node or tool and surfaces the query to the human.
- ✦ `Command(resume=...)` : sends the human's response back in and resumes the paused run.
- ✦ **Human-assistance tool** → an ordinary `@tool` that calls `interrupt`. The LLM chooses it based on its docstring ("Request assistance from a human").
- ✦ Requires a checkpoint, because the paused state has to be saved.

Flow:



Code essentials: `from langgraph.types import Command,`
`interrupt`

```

@tool
def human_assistance(query: str) → str:
    """Request assistance from a human."""
    resp = interrupt({"query": query})
    return resp["data"]

graph.stream(Command(resume={"data": "Use LangGraph..."}), config, stream_mode="value
  
```

watch out

→ Which tool gets called depends on wording. The first prompt went to Tavily. Adding "assistance" so it matched the docstring made the LLM call `human_assistance`.

→ You can interrupt and resume as many times as you need in one conversation.



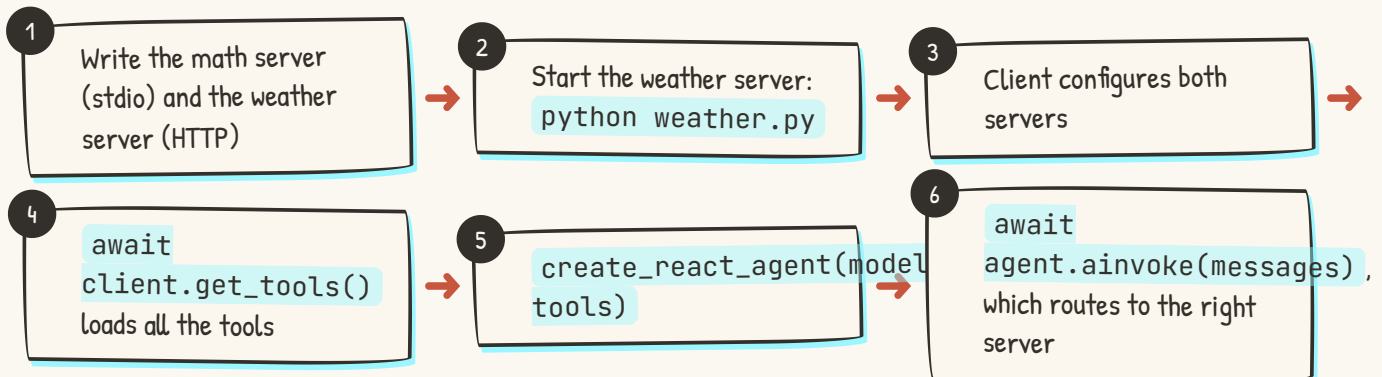
Big idea: Put tools behind MCP servers built with FastMCP, then use one

`MultiServerMCPClient` to load all their tools into a LangGraph ReAct agent. The demo has two servers on different transports: stdio and streamable HTTP.

Key concepts:

- ✦ **MCP server** → exposes tools, context and prompts.
- ✦ **MCP client** → keeps a 1:1 connection with a server inside the host app.
- ✦ **Host app** → Claude Desktop or your own chatbot.
- ✦ **langchain-mcp-adapters** : connects MCP tools to LangChain/LangGraph.
- ✦ FastMCP (`mcp` package): a Pythonic way to define servers. Each `@mcp.tool()` function is a tool, and its docstring is what the LLM reads.
- ✦ **stdio transport**: the server talks over standard input/output. The client launches it as a subprocess. Good for local testing. Running it by hand appears to do nothing.
- ✦ **streamable-http transport**: the server runs as an HTTP service (default `localhost:8000`, endpoint `/mcp`). This is the shape you would deploy remotely.
- ✦ **uv workflow** → `uv init` → `uv venv` → activate → `uv add -r requirements.txt` (requirements: `langchain-groq`, `langchain-mcp-adapters`, `mcp`, `langgraph`).

Flow:



Compare:

	<i>stdio</i>	<i>streamable-http</i>
How it runs	Subprocess launched by the client (command + args)	Standalone HTTP API
Client config	<code>command</code> , <code>args</code> (absolute path), <code>transport</code>	<code>url</code> (.../mcp), <code>transport</code>
Typical use	Local dev and testing	Remote or production services

Code essentials:

```
mcp = FastMCP("Math")          # from mcp.server.fastmcp import FastMCP
@mcp.tool()
def add(a: int, b: int) → int:
    """Add two numbers"""
    return a + b
if __name__ == "__main__":
    mcp.run(transport="stdio")    # weather: "streamable-http"
    client = MultiServerMCPClient({"math": {"command": "python", "args": ["math_server.py",
    "weather": {"url": "http://localhost:8000/mcp", "transport": "streamable-http"}}})
    tools = await client.get_tools(); agent = create_react_agent(ChatGroq(model="qwen-qwq
```

Run it with `asyncio.run(main())`. Read the answer from `resp["messages"][-1].content`.

⚠ watch out

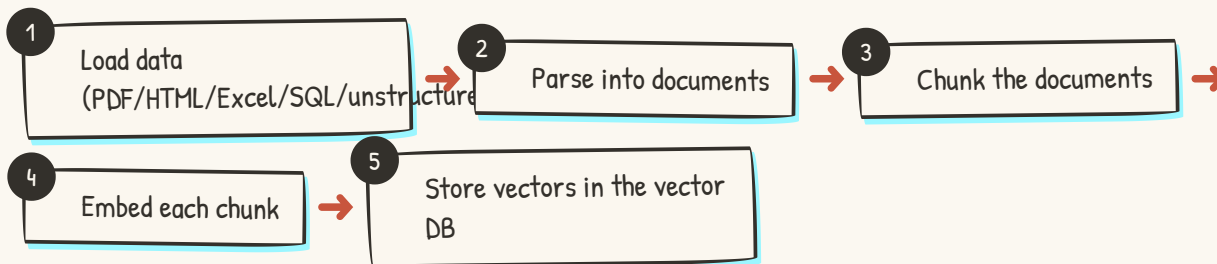
- Give the stdio server script an absolute path in the client config.
- The HTTP server has to be running before the client starts.
- Put `GROQ_API_KEY` in `.env` and load it with `load_dotenv`.
- Transient errors happen sometimes. Restarting fixes them.
- The LLM may add its own commentary to a tool's output. The demo's hardcoded "always raining in California" was followed by an explanation that California's climate actually varies.

💡 **Big idea:** RAG makes an LLM answer from an external knowledge base (a vector DB of your own data) instead of retraining it. This deals with outdated knowledge and with private or changing data more cheaply than fine-tuning.

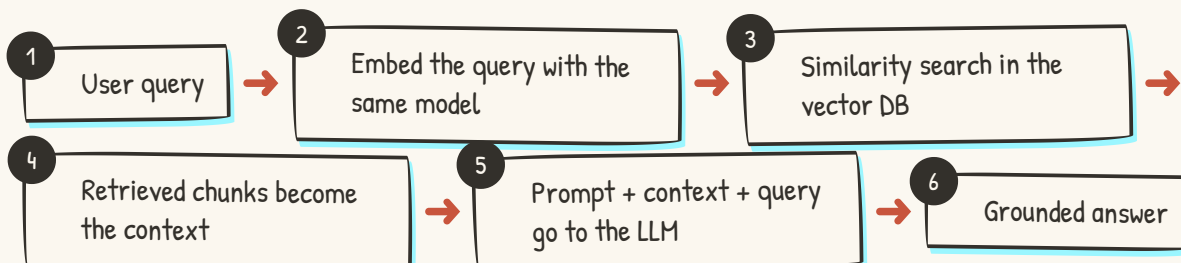
Key concepts:

- ♦ **Hallucination** → when asked about events after its training cutoff, the LLM makes up a confident-sounding answer.
- ♦ **Private data problem** → company HR and finance policies are not in the LLM, and fine-tuning billions of parameters is expensive and slow, especially when the data changes often.
- ♦ **Vector DB / vector store** → holds embeddings of document chunks and supports similarity search (e.g. cosine).
- ♦ **Embeddings** → text turned into vectors. Options include OpenAI, Google Gemini, Hugging Face and other open-source models, at different costs.
- ♦ **Context** → the retrieved chunks passed to the LLM.
- ♦ **R-A-G** → **R**etrieval (fetch chunks), **A**ugmentation (context + prompt), **G**eneration (LLM output).
- ♦ Perplexity is an example of a RAG-style product (retrievers + web search + LLM summarization).

Flow (ingestion pipeline):



Flow (retrieval pipeline):



Compare:

<i>Approach</i>	<i>Cost</i>	<i>Handles changing data</i>	<i>Hallucination</i>
Plain LLM + prompt	Low	No	High on unknown or recent facts
Fine-tuning	High, slow	Poorly (needs retraining)	Reduced for the trained domain
RAG	Low	Yes (re-ingest)	Reduced, not eliminated

! *watch out*

- RAG reduces hallucination but does not remove it. If the answer is not in the DB, the LLM can still make one up.
- Parsing and chunking quality has the biggest effect on retrieval quality.
- The course later covers chunking strategies (e.g. semantic chunking), context engineering and agentic RAG.

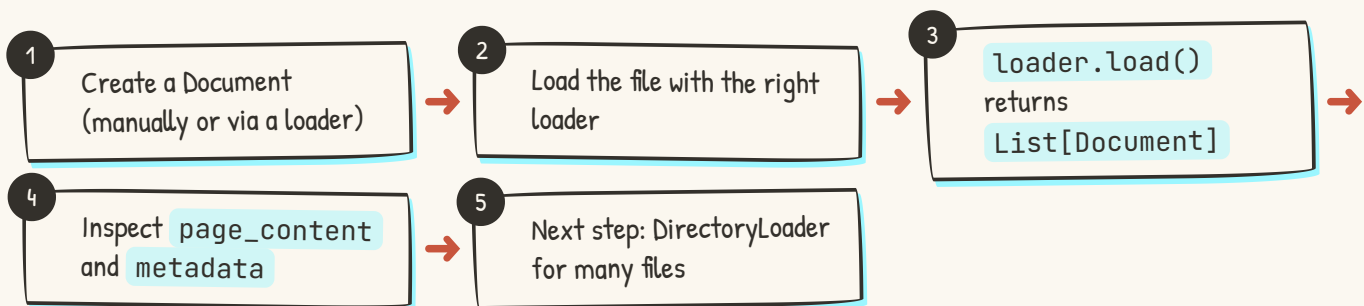


Big idea: Every loader converts its source into LangChain Document objects (page_content + metadata). That common format is what chunking, embedding and vector storage work on, so you need to understand it first.

Key concepts:

- ◆ **Document**: page_content holds the actual text. metadata is a dict (source, pages, author, date created, ...).
- ◆ **Why metadata matters** → it lets you filter during similarity search, e.g. restrict results to author = X.
- ◆ **Why chunk** → embedding models and LLMs have fixed context limits, so a 100-page PDF cannot be embedded in one piece.
- ◆ **Loaders** → TextLoader, PyPDFLoader / PyMuPDF, CSVLoader, WebBaseLoader, DirectoryLoader. All of them return a list of Document s and fill in metadata automatically (e.g. source).
- ◆ **Setup** → uv init, uv venv, then install langchain, langchain-core, langchain-community, pypdf, pymupdf, ipykernel. Folders: data/, notebook/.

Flow:



Code essentials:

```
from langchain_core.documents import Document
doc = Document(page_content="Main text used to build RAG",
               metadata={"source": "example.txt", "pages": 1, "author": "Krish", "date": "2023-10-27"})
from langchain_community.document_loaders import TextLoader # older/alt import path
docs = TextLoader("../data/text_files/python_intro.txt", encoding="utf-8").load()
```

⚠ watch out

- LangChain import paths keep moving (`langchain.document_loaders` vs `langchain_community.document_loaders`). Use whichever one does not give a deprecation warning.
- Relative paths from inside `notebook/` need `../`.
- Pass `encoding="utf-8"` to the text loader.
- **Assignment** → build the same ingestion pipeline for other formats (CSV, Excel, JSON).

📝 ★ Part summary —

- ◆ Sending the ToolNode output back to the LLM instead of to END turns a one-shot tool caller into a **ReAct agent** (act, observe, reason) that can handle multi-step, multi-tool requests.
- ◆ LangGraph **memory** comes from `MemorySaver` passed as `checkpointer=` at compile time, plus a per-session `thread_id` in `config["configurable"]`.

Streaming: `updates` gives only each node's change, `values` gives the full accumulated state, and `astream_events` gives detailed debugging events. **Human-in-the-loop** uses `interrupt()` to pause and `Command(resume=...)` to continue, and needs a `checkpointer`.

MCP: define tools with `FastMCP ( @mcp.tool() )`, serve them over `stdio` (local subprocess) or `streamable-http` (`localhost:8000/mcp`), and load them through `MultiServerMCPClient.get_tools()` into `create_react_agent`.

RAG: has two pipelines. Ingestion: load, parse, chunk, embed, store. Retrieval: embed the query, run similarity search, add the context to the prompt, generate. It all rests on LangChain `Document` objects (`page_content` + `metadata`), and the `metadata` enables filtered retrieval.



Part 4 · 5:46 – 7:36

RAG Pipeline + Vectorless RAG

 **Big idea:** LangChain loaders read files from many sources (text, PDF, S3, Excel, DB, ...) and turn each one into the same `Document` structure (`page_content` + `metadata`). That shared shape is what lets chunking work on any source.

Key concepts

- ◆ **DirectoryLoader** : loads every file in a folder that matches a glob pattern (e.g. `**/*.txt`). `loader_cls` picks the loader used for each file.
- ◆ **show_progress=True** : needs the `tqdm` package. Set it to `False` if you don't want to install it.
- ◆ **PyPDFLoader** vs **PyMuPDFLoader** : both live in `langchain_community.document_loaders` . The speaker says PyMuPDF is generally better.
- ◆ **PDF metadata** → you get richer fields for free, such as `source` , `file_path` , `total_pages` , `creationdate` , `author` and `format` .
- ◆ One `Document` per PDF page. A 15-page PDF gives 15 Documents.

Code essentials

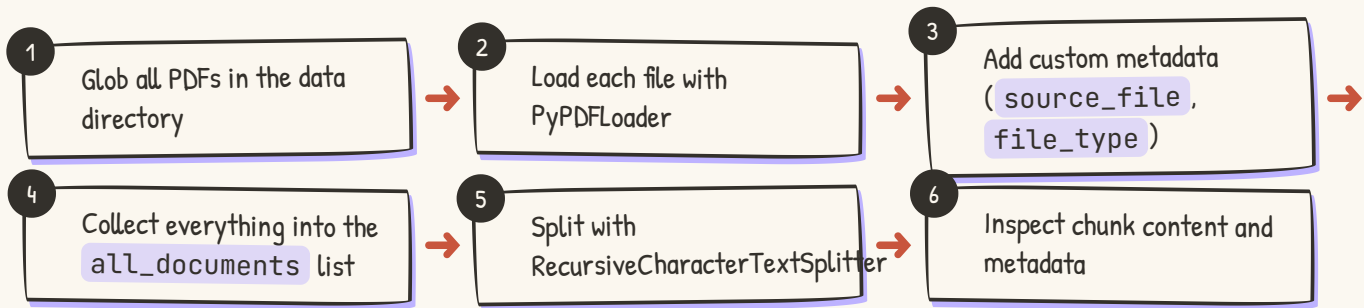
```
from langchain_community.document_loaders import DirectoryLoader, PyMuPDFLoader
loader = DirectoryLoader("data/pdf", glob="**/*.pdf",
                        loader_cls=PyMuPDFLoader, show_progress=False)
pdf_docs = loader.load() # list[Document]
```

watch out

- Passing `encoding` through to `PyMuPDFLoader` raises an error (`get_text` got an unexpected argument). Leave it out for PDFs. It is only needed for `TextLoader` .
- **Homework** → load Excel, DB and S3 sources yourself using the LangChain loader docs. S3 needs authentication set up first.

 **Big idea:** Build the ingestion half of RAG step by step in a notebook: read every PDF, add your own metadata, then split the pages into overlapping chunks.

Flow



Key concepts

- ✦ `process_all_pdfs(dir)` : helper function. It uses `Path.glob` to find PDFs, loads each one, adds metadata and returns the combined list.
- ✦ `split_documents(docs, chunk_size=1000, chunk_overlap=200)` : wraps `RecursiveCharacterTextSplitter`.
- ✦ **Chunk overlap** → characters shared between neighbouring chunks, so meaning is kept across a split.
- ✦ **Separators** → an ordered list such as `["\n\n", "\n", " ", ""]`. The splitter falls back from paragraph to line to word to character.
- ✦ **Result in the demo** → 64 page Documents became 359 chunks.

Code essentials

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200,
                                          length_function=len, separators=["\n\n", "\n", " ", ""])
chunks = splitter.split_documents(all_pdf_documents)
```

watch out

- You can add as many custom metadata keys as you like. They are carried into each chunk and later into the vector DB.





EmbeddingManager and VectorStore Classes (ChromaDB)

6:03-6:21

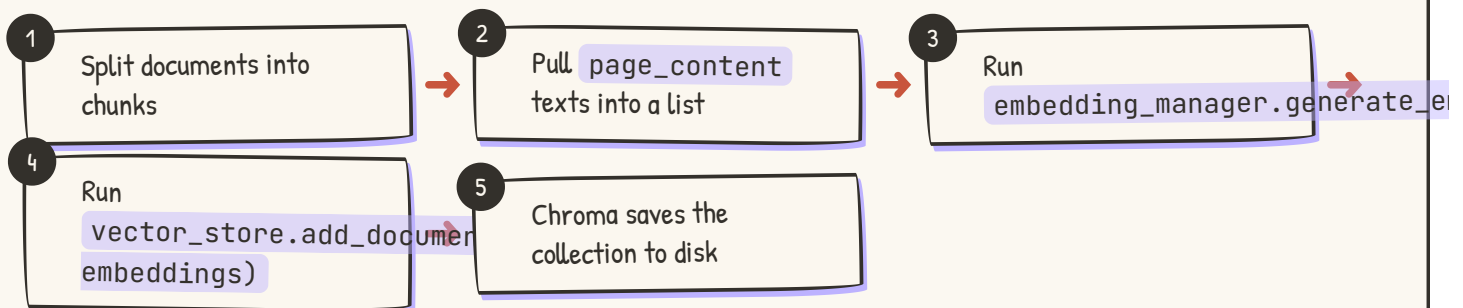


Big idea: Wrap embedding and storage in separate classes so each pipeline stage connects cleanly to the next. Sentence-Transformers turns text into vectors and a persistent ChromaDB collection stores them.

Key concepts

- ◆ **all-MiniLM-L6-v2** : Hugging Face sentence-embedding model that outputs **384-dimension** vectors.
- ◆ **EmbeddingManager** : `__init__(model_name)` calls `_load_model()`, which loads `SentenceTransformer`. `generate_embeddings(texts)` returns `model.encode(texts, show_progress_bar=True)` as a numpy array.
- ◆ **_load_model** : the leading underscore marks it as a protected, internal method.
- ◆ **get_sentence_embedding_dimension()** : reports the vector size (384).
- ◆ **VectorStore** : `__init__(collection_name="pdf_documents", persist_directory="data/vector_store")`, then `_initialize_store()`.
- ◆ **chromadb.PersistentClient(path=...)** : client that saves the DB to disk so it can be reloaded later.
- ◆ **get_or_create_collection(name, metadata=...)** : a collection is the named container that holds the vectors.
- ◆ **add_documents(documents, embeddings)** : builds parallel lists of ids (`uuid`), metadatas (plus `doc_index` and `content_length`), document texts and embedding lists, then calls `collection.add(...)`.
- ◆ **Requirements** → `sentence-transformers`, `chromadb`, `faiss-cpu`, `scikit-learn` (for cosine similarity), `numpy`. Installed with `uv add -r requirements.txt`.

Flow



Code essentials

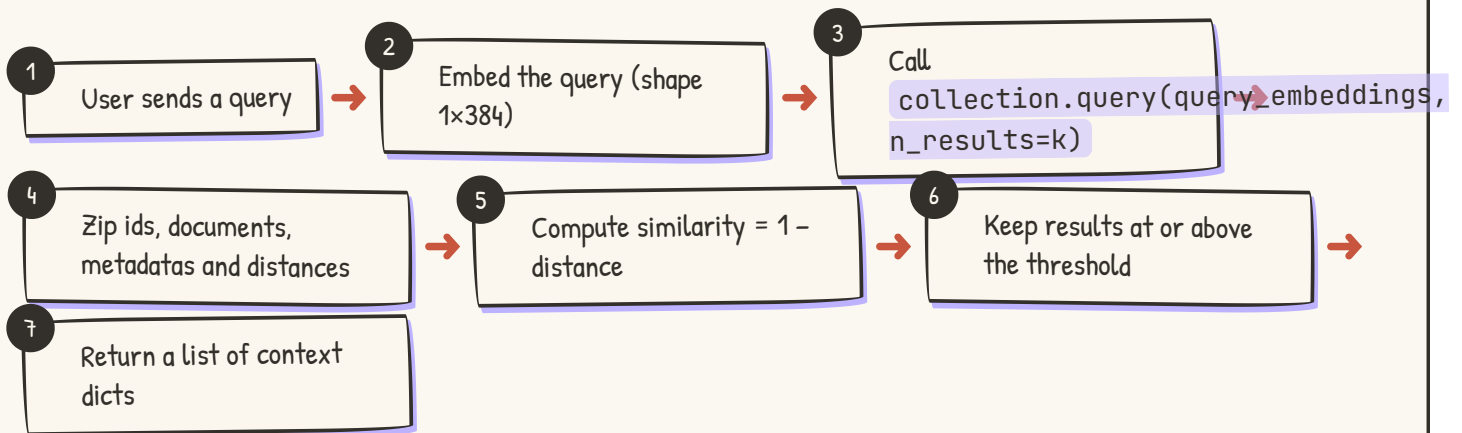
```
texts = [doc.page_content for doc in chunks]
embeddings = embedding_manager.generate_embeddings(texts)
vector_store.add_documents(chunks, embeddings)
# inside add_documents:
self.collection.add(ids=ids, embeddings=emb_list,
                    metadatas=metadatas, documents=doc_texts)
```

⚠ watch out

- `add_documents` should check that `len(documents) == len(embeddings)` before inserting.
- Convert numpy embeddings with `.tolist()` before passing them to Chroma.
- Every record needs a unique ID. Use `uuid` and prefix it, e.g. `doc_<uuid>_<i>`.
- The collection count should match the number of chunks: 0 before insert, 359 after.

 **Big idea:** A retriever is a simple interface on top of the vector store. It embeds the query with the same model, runs a similarity search and returns the top-k chunks with their scores.

Flow



Key concepts

- ✦ `RAGRetriever(vector_store, embedding_manager)` : holds references to both earlier objects, so the pipelines are linked.
- ✦ `retrieve(query, top_k=5, score_threshold=0.0)` : returns a list of dicts with `id`, `content`, `metadata`, `similarity_score`, `distance` and `rank`.
- ✦ **Chroma result keys** → `documents`, `metadatas`, `distances`, `ids`. Each is nested one level per query.

Code essentials

```
q_emb = self.embedding_manager.generate_embeddings([query])[0]
res = self.vector_store.collection.query(
    query_embeddings=[q_emb.tolist()], n_results=top_k)
for i,(id_,doc,meta,dist) in enumerate(zip(res["ids"][0], res["documents"][0],
                                          res["metadatas"][0], res["distances"][0])):
    score = 1 - dist
```

 **watch out**

- You must use the same embedding model at ingestion time and at query time.
- So far this only returns context. Generation (the LLM step) comes next.

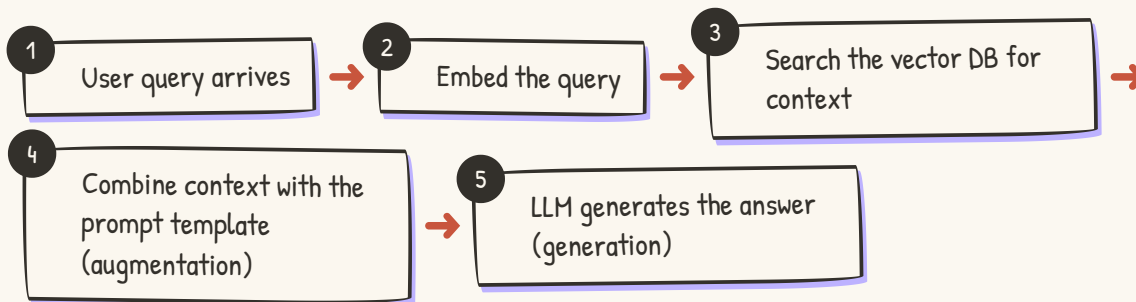
Augmentation + Generation with Groq LLM: Simple, Enhanced, Advanced RAG

6:32-6:49



Big idea: RAG means Retrieval (query → vectors → vector DB → context), then Augmentation (context + prompt), then Generation (LLM answer). The speaker shows three versions of increasing richness.

Flow (query pipeline)



Key concepts

- ◆ **ChatGroq** (from `langchain_groq`): Groq-hosted LLM. Demo settings: model `gemma2-9b-it`, `temperature=0.1`, `max_tokens=1024`. API key comes from `.env` via `python-dotenv`.
- ◆ **`rag_simple(query, retriever, llm, top_k=3)`**: joins the retrieved content with `"\n\n"`, returns "No relevant context found" if nothing comes back, then calls `llm.invoke(prompt)` and returns `response.content`.
- ◆ **`rag_advanced(query, retriever, llm, top_k, min_score, return_context)`**: returns `answer`, `sources` (source file, page, score, 300-character preview), `confidence` (taken from the similarity scores) and, optionally, the full context.
- ◆ **Third, advanced pipeline** → adds streaming, citations, query history and summarization. Left to the viewer to read.

Compare

Variant	Returns	Extras
Simple RAG	answer text	none

Variant	Returns	Extras
Enhanced RAG	answer, sources, confidence, optional context	page and file citations, min score filter
Advanced RAG	answer plus citations	streaming, history, summarization

Code essentials

```
from langchain_groq import ChatGroq
llm = ChatGroq(groq_api_key=key, model_name="gemma2-9b-it", temperature=0.1, max_token_limit=1024)
results = retriever.retrieve(query, top_k=3)
context = "\n\n".join(d["content"] for d in results)
prompt = f"Use the following context to answer concisely.\nContext:\n{context}\nQuestion: {query}\nAnswer:"
answer = llm.invoke(prompt).content
```

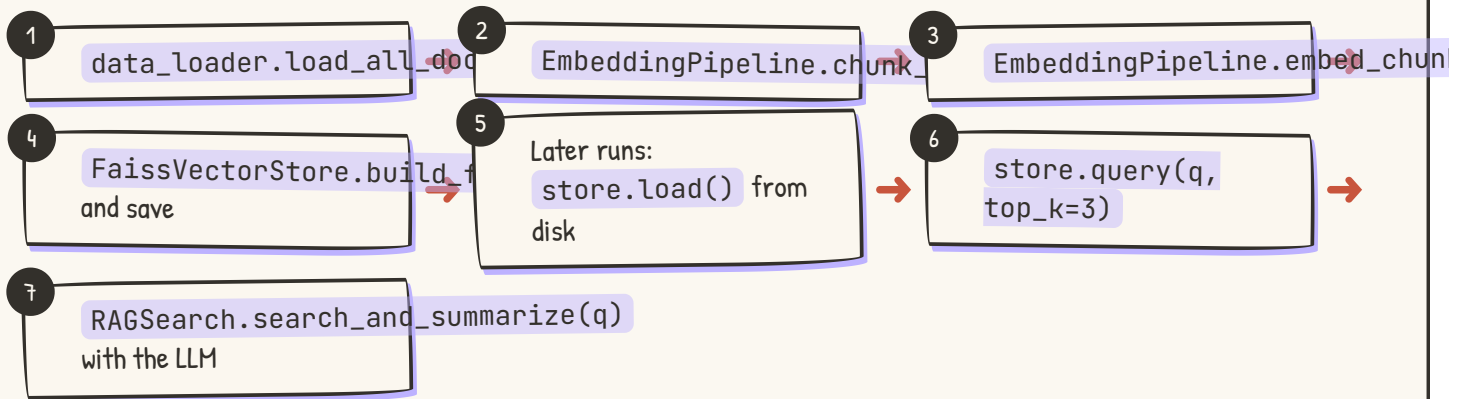
⚠ watch out

- When testing, if `os.getenv` fails, paste the key directly to rule out env-loading problems. Don't leave it hardcoded.
- Some queries return no relevant context even after lowering `min_score` to 0.1. Treat that as a reason to tune retrieval (chunk size, threshold), which comes later in the course.

Modular RAG Project Structure (src/) with FAISS 6:49-7:10

 **Big idea:** Move the notebook code into a reusable package with one file per pipeline stage, wired together from `app.py`. This version uses FAISS for local persistence.

Flow



Key concepts

- ✦ `src/__init__.py`, `data_loader.py`, `embedding.py`, `vectorstore.py`, `search.py`, plus `app.py` at the root.
- ✦ `load_all_documents(data_dir) → List[Any]`: globs `**/*.pdf`, loads with `PyPDFLoader` and extends a list. Homework: add TXT, CSV, SQL and other loaders.
- ✦ `EmbeddingPipeline(model="all-MiniLM-L6-v2", chunk_size=1000, chunk_overlap=200)`: has `chunk_documents()` and `embed_chunks()`.
- ✦ `FaissVectorStore(persist_dir="faiss_store")`: has `build_from_documents`, `add_embeddings (IndexFlatL2)`, `save` (writes `faiss.index` and `metadata.pkl`), `load`, `search` and `query` (encodes the query as `float32`).
- ✦ `RAGSearch`: loads the FAISS store, retrieves documents and metadata, builds a prompt, calls the Groq LLM and returns a summary.

Compare

	Notebook (Chroma)	Modular (FAISS)
Storage	<code>chromadb.PersistentClient</code> collection	<code>faiss.index</code> + <code>metadata.pkl</code>
Index	Chroma-managed	<code>IndexFlatL2</code>

Structure cells and classes in one notebook

src/ package + app.py

Code essentials

```
from src.data_loader import load_all_documents
from src.vectorstore import FaissVectorStore
from src.search import RAGSearch
store = FaissVectorStore("faiss_store")
# store.build_from_documents(load_all_documents("data")) # first run only
store.load()
print(RAGSearch().search_and_summarize("What is attention mechanism?", top_k=3))
```

! watch out

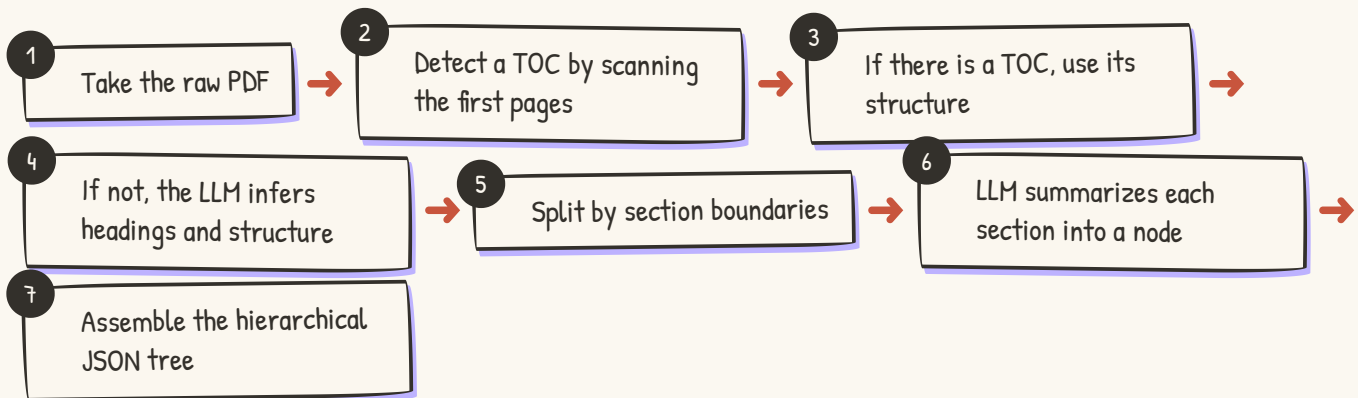
- `load_all_documents` has to `return documents`. The first version forgot to.
- Build the index once and just `load()` it afterwards. Only rebuild when new documents arrive.
- The speaker also mentions Typesense (covered earlier) as another open-source search/vector store.

💡 **Big idea:** Vectorless RAG drops chunking, embeddings and the vector DB. An LLM builds a hierarchical JSON "tree index" of the document's sections, each with a summary. At query time the LLM reasons over that tree, the way a person uses a table of contents, picks the relevant nodes and answers with citations.

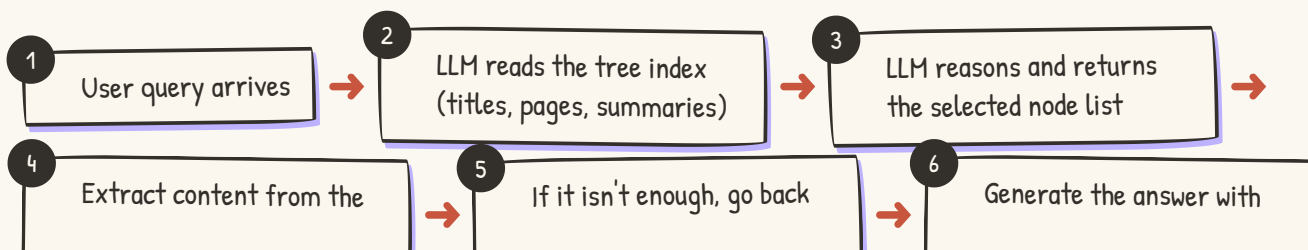
Key concepts

- ♦ **Traditional vector RAG** → PDF → chunk → embed → vector DB → similarity search → flat chunks → LLM.
- ♦ **LLM tree builder** → builds a hierarchy of sections (e.g. Introduction → AI → ML / DL). Each node holds `node_id`, `title`, page range and an LLM-written summary of that section.
- ♦ **JSON tree index** → the serialized tree. It is passed to the LLM as context.
- ♦ **Section-aware splitting** → splits follow logical boundaries rather than token counts, so a section isn't cut into pieces where only one part gets retrieved.
- ♦ **Reasoning-based retrieval** → the LLM navigates the tree the way a human expert would.
- ♦ **PageIndex (Vectify AI, [pageindex.ai](https://chat.pageindex.ai))** → repository and API that implement this. There is a chat UI at chat.pageindex.ai. It is free up to a request limit, not fully open source.

Flow (indexing)



Flow (retrieval)



Compare

Aspect	Traditional vector RAG	Vectorless (PageIndex) RAG
Preprocessing	chunk + embed	LLM builds a section tree with summaries
Storage	vector DB	JSON tree index (no DB)
Retrieval	cosine similarity top-k	LLM reasons over the tree
Split unit	token or character count	logical sections
Output context	flat chunks	section, title, pages, summary
Citations	chunk metadata	section + page citations

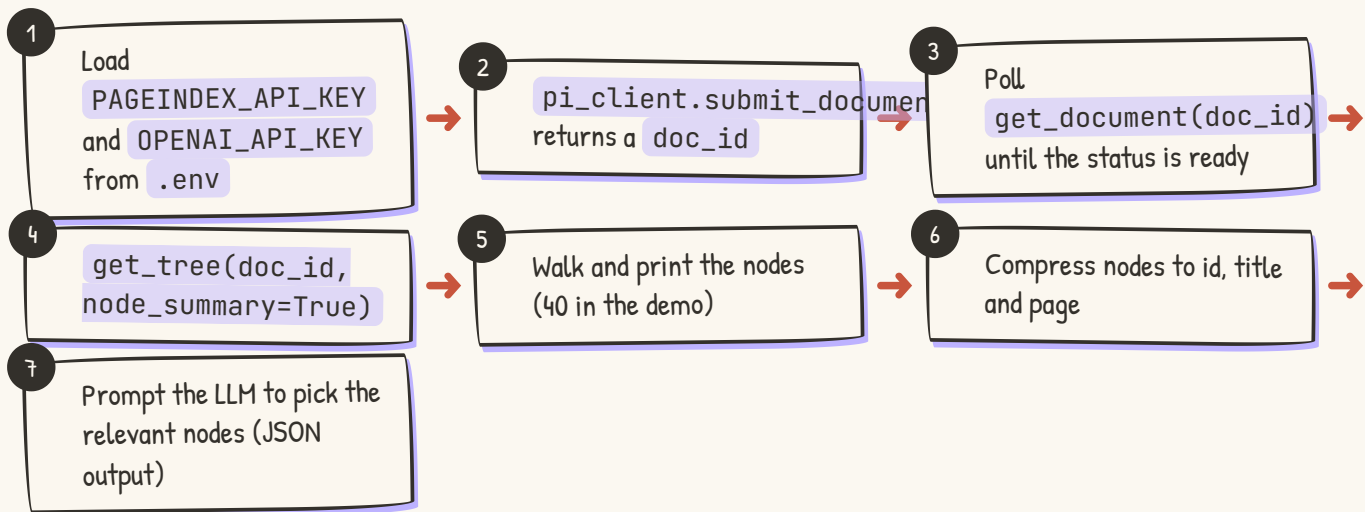
⚠ watch out

- Plain chunking can split one section into several chunks. If only some of them are retrieved, the answer is incomplete. Tree nodes keep whole sections together.
- You can build this yourself with any LLM (e.g. Claude) once you understand the concept. PageIndex just packages it.



Big idea: Upload a PDF to PageIndex, wait for the tree to build in the background, inspect it, then run an "LLM tree search" by giving OpenAI the query plus the compressed tree and asking which nodes hold the answer.

Flow



Key concepts

- ◆ `PageIndexClient(api_key=...)` (from `pageindex import PageIndexClient`) and `OpenAI(api_key=...)`.
- ◆ Tree building is asynchronous. A roughly 50-page PDF takes about 30–90 seconds.
- ◆ **Node fields** → `title`, `node_id` (e.g. `0000`), `page_index`, `text` / `summary`, and child `nodes`.
- ◆ **Demo document** → a 45–48 page course syllabus with a TOC but no page numbers. PageIndex still recovered 24 top-level sections and 40 nodes in total.
- ◆ **Tree-search prompt** → "Given the query and the document tree (like a TOC), identify the nodes most likely to contain the answer; think step by step." The tree is passed in as JSON and the call goes to OpenAI chat completions.

Code essentials

```
from pageindex import PageIndexClient
from openai import OpenAI
pi = PageIndexClient(api_key=os.getenv("PAGEINDEX_API_KEY"))
```

```
doc_id = pi.submit_document("sample_document.pdf")["doc_id"]
tree = pi.get_tree(doc_id, node_summary=True).get("result")
```

! watch out

- Save the `doc_id`. Every later call in the notebook uses it.
- Prefer text-heavy PDFs for testing.
- **Packages** → `pageindex`, `openai`, `python-dotenv`. The free tier covers roughly 1,000 documents.

✏️ ★ Part summary —

- ◆ Any source (PDF, TXT, directory, S3) becomes a LangChain `Document` through loaders. `RecursiveCharacterTextSplitter` (1000 / 200 in the demo) turned 64 pages into 359 chunks.
- ◆ The ingestion pipeline is built from classes: `EmbeddingManager` (all-MiniLM-L6-v2, 384 dimensions) feeds `VectorStore` (a persistent ChromaDB collection with uuid IDs and extra metadata).
- ◆ **RAGRetriever** embeds the query, calls `collection.query`, turns distance into a similarity score (1 - distance), applies a threshold and returns top-k context. A Groq LLM then does augmentation and generation in simple, enhanced and advanced variants.
- ◆ The same pipeline was refactored into a `src/` package (data_loader, embedding, vectorstore with FAISS `IndexFlatL2` + pickle metadata, search) driven by `app.py`, with build-once, load-later persistence.
- ◆ Vectorless RAG (PageIndex) replaces chunks, embeddings and the vector DB with an LLM-built hierarchical JSON tree of summarized sections. The LLM reasons over the tree to pick nodes, keeping section boundaries intact and giving page-level citations.



Part 5 · 7:36 – 9:28

Deep Agents + Guardrails



Vectorless RAG: Querying the Tree and Generating Answers

7:36-7:40

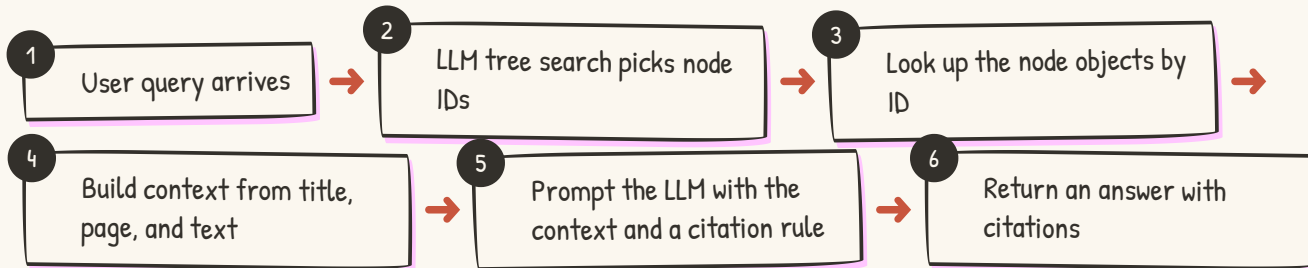


Big idea: An LLM walks a JSON tree index of the document to pick relevant nodes. The text of those nodes then becomes the context for answering. No embeddings and no vector DB are involved.

Key concepts:

- ♦ **LLM tree search** → the LLM reads the node titles and summaries and returns the IDs of the relevant nodes (e.g. 0011, 0012, ...).
- ♦ **find_nodes_by_id** : walks the tree recursively and collects the nodes whose IDs match.
- ♦ **generate_answer** : builds the context from each node's section, title, page index and text, then asks the LLM to answer from that context only.
- ♦ **Citation prompt** → "expert document analyst". Every claim must cite the section title and page number in parentheses.

Flow:



Code essentials: `llm_tree_search(query, page_index_tree)`,

`find_nodes_by_id(tree, ids)`, `generate_answer(query, nodes)`

(returns "No relevant section found" when no nodes match), OpenAI chat call.



watch out

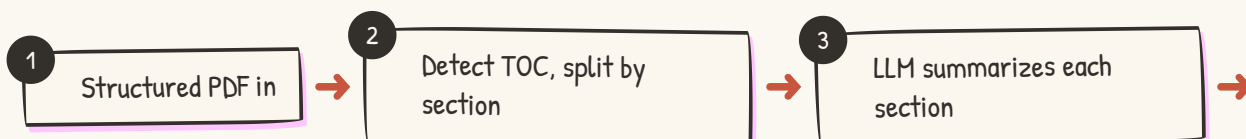
- If the search returns no nodes, return an explicit "not found" instead of letting the LLM guess.
- **Open question** → where to keep a large tree. This is answered in the next section.

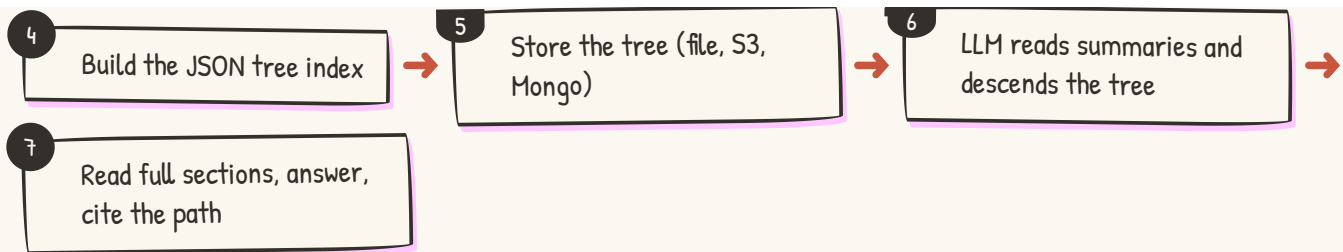
💡 **Big idea:** Traditional RAG gives you scale and cheap lookups. Vectorless (tree / PageIndex) RAG gives you reasoning, preserved structure and explainability. They complement each other, and production systems are moving to hybrid.

Key concepts:

- ♦ **Traditional pipeline** → chunk, embed, store in a vector DB (Pinecone, Chroma, FAISS, Qdrant, Weaviate), embed the query, run cosine similarity, put the top chunks into the LLM prompt.
- ♦ **Vectorless pipeline** → a structured PDF goes into an LLM tree builder, which produces a JSON tree index (a hierarchy of sections; leaf nodes hold the title, page range and summary). The LLM traverses the tree at query time.
- ♦ **Building the tree** → detect a table of contents (TOC). If one exists, parse the chapters and split by section, following logical boundaries rather than token counts. Then summarize each section and assemble a parent/child/grandchild tree. Without a TOC, the pages are processed directly.
- ♦ **Storing the tree** → it is just JSON, so it can live on the file system, in S3, in MongoDB, or in any key-value or JSON store.
- ♦ Weaknesses of traditional RAG:
 - ♦ **Chunking destroys context** → related information split across chunks can be missed.
 - ♦ **Similarity is not relevance** → embeddings can confidently match the wrong text.
 - ♦ No cross-section reasoning (e.g. "compare risk vs mitigation").
 - ♦ **Poor explainability** → a cosine score is not a reason.
 - ♦ **Embedding drift** → changing the embedding model means re-embedding everything.
- ♦ **Strengths of vectorless RAG** → whole sections stay intact, the LLM can reason across sections, retrieval is explainable (it returns the navigation path), and there is no embedding pipeline.
- ♦ Weaknesses of vectorless RAG:
 - ♦ Several LLM calls per query, so latency runs from hundreds of milliseconds to seconds.
 - ♦ Scales to roughly 10 to thousands of documents, not millions.
 - ♦ Needs structured documents.
 - ♦ Tooling (PageIndex) is still immature.

Flow (vectorless):





Compare

Dimension	Traditional RAG	Vectorless RAG
Scale	Millions of docs	10s-1000s of docs
Latency	~ms	100s of ms to seconds
Cost/query	Cheap (1 embedding + 1 search)	Higher (multiple LLM calls)
Cross-section reasoning	Weak	Strong
Explainability	Cosine score	Navigation path
Best for	Factoid Q&A over mixed corpora	Long structured documents
Setup	Embedding pipeline + vector DB	Tree builder only
Ecosystem	Mature	Emerging

! watch out

- **Use traditional RAG for** → huge mixed corpora (blogs, tickets, transcripts, KB articles), latency-critical chatbots and search, short factoid questions ("who is the CEO?"), and cost-sensitive workloads with thousands of queries per minute.
- **Use vectorless RAG for** → annual reports, 10-Ks, legal contracts and textbooks; cases where reasoning matters more than similarity; compliance, audit, legal or financial work where you must "show your work"; and documents where chunking destroys the meaning.
- **Rule of thumb** → let the documents decide, not the hype. Use a hybrid when the data mixes both kinds.

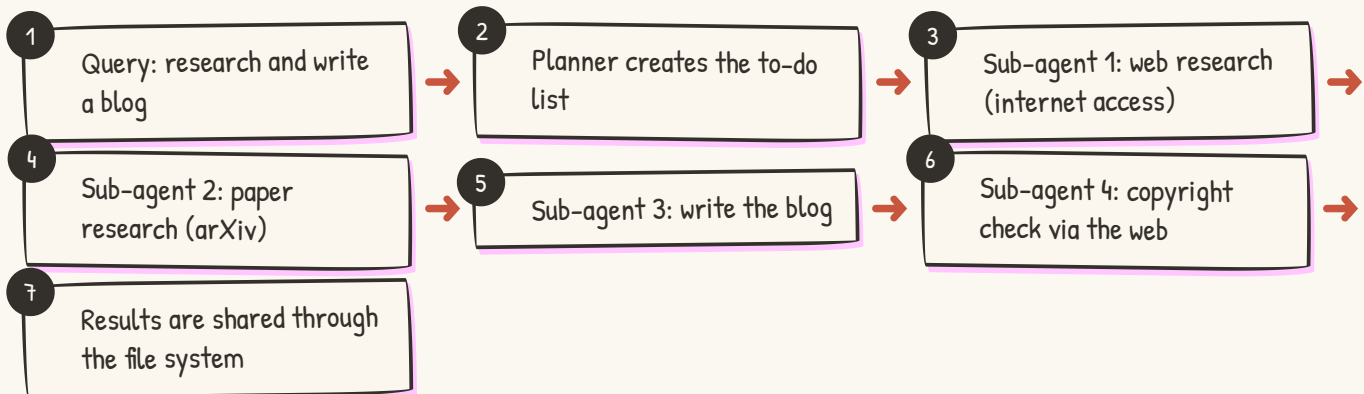


Big idea: "Shallow" agents (an LLM plus tools in a simple loop, or ReAct) cannot plan, break down complex queries, or keep context. Deep agents (ChatGPT/Claude deep research, Claude Code, Manus) add planning, sub-agents, a detailed system prompt and a shared file system.

Key concepts:

- ♦ **Shallow agent** → input goes to the LLM, which either answers or calls one tool and returns. There is no explicit planning, complex queries are not split, and context retention is limited.
- ♦ **ReAct agent** → the LLM has many tools (Wikipedia, search, etc.) and loops through reason, act, observe until done (e.g. "2+2 then x5"). It improves on the basic agent but is still shallow: no structured plan, no state management, no persistent memory.
- ♦ Deep agent, four core components: 1. Planning tool: writes a to-do list that breaks the task into steps (e.g. a 3-night/4-day Paris trip on a ₹100k budget, planned day by day). 2. Sub-agents: one per to-do item, each specialized and able to run in parallel. 3. System prompt: a long, detailed spec for behavior, tone and style. Claude Code's system prompt is the example given. 4. File system: persistent shared memory that all sub-agents can read and write. This is how they hand off work.

Flow (blog-writing example):



Compare

	Shallow / ReAct	Deep agent
Planning	None	Explicit to-do list
Decomposition	No	Sub-agents per task

<i>Shallow / ReAct</i> <i>Deep agent</i>		
Memory	Limited context	File system (persistent)
Complex multi-step tasks	Poor	Designed for them

💡 **Big idea:** The `deepagents` library is built on `LangGraph` and inspired by `Claude Code`, `Manus` and `OpenAI deep research`. `create_deep_agent` looks like `LangChain's create_agent` but comes with middleware already attached for to-dos, the file system, summarization and sub-agents.

Key concepts:

- ♦ **Setup** → `uv init`, then `uv venv`, activate, `uv pip install -r requirements.txt`.
- ♦ **Packages** → `deepagents` (pulls in `langgraph`), `langchain`, `langchain-openai`, `langchain-groq`, `ipykernel`, `tavily-python`, `python-dotenv`.
- ♦ Keys in `.env`: `OpenAI`, `Groq`, `Google`, `Tavily`. `Tavily` provides real-time web search.
- ♦ **Tool** → a `web_search` function wrapping `TavilyClient.search`. Its parameters follow `Tavily's API`: `query`, `max_results=5`, `topic: Literal["general", "news", "finance"]`, `include_raw_content=False`.
- ♦ **Model** → `init_chat_model(...)` (the demo uses a `Groq-hosted Qwen 32B`), or `ChatOpenAI`, `ChatGroq` or `ChatGoogleGenerativeAI`.
- ♦ Built-in middleware (visible when you display the graph): a to-do list hook after the model, a summarization hook, and patching of tool calls. Simple agents only get nodes and edges unless you add middleware yourself.
- ♦ **Output** → `result["messages"][-1].content` holds the answer.
`result["files"]` holds files the agent wrote (large tool results, queries, URLs, timestamps) so context survives summarization.

Code essentials:

```
from deepagents import create_deep_agent
from langchain.chat_models import init_chat_model
model = init_chat_model("groq:qwen-...") # any provider
agent = create_deep_agent(tools=[web_search],
                          system_prompt="Act as a researcher", model=model)
res = agent.invoke({"messages":[{"role":"user","content":"What is deep agents?"}]}))
print(res["messages"][-1].content); res["files"]
```

⚠ *watch out*

- The keyword arguments are `model=` and `tools=`, not `models=` or `tool=`.
- Check the parameter names against the Tavily docs (the speaker also says to keep their order).
- Deep agents are slow because they do a lot of research, so use streaming.
- Part 2 (not covered here) adds customization: backends, sub-agents and interrupts.



Big idea: Guardrails are safety layers around an agent pipeline. They make sure the agent only takes safe input, only performs approved actions, and only returns validated, compliant output. LangChain implements them as middleware hooks.

Key concepts:

- ◆ **Placement** → on the input (e.g. block "how to hack a server" before the LLM), around tool calls, and on the output.
- ◆ **Deterministic approach** → rules such as regex and keyword matching.
- ◆ **Model-based approach** → an LLM classifies content as safe or unsafe.
- ◆ LangChain middleware types: 1. PII middleware: detects email, credit card, IP, MAC address, URL. Can apply to input, output and tool calls. 2. Human-in-the-loop middleware: pauses before sensitive tools until a human approves or rejects. Needs threads and a checkpoint. 3. Before-agent hook: runs before any LLM call. Blocked requests cost nothing and jump straight to the end. 4. After-agent hook: checks the final response before the user sees it. Can replace or mutate unsafe content and can use a cheap or small model. 5. Layered guardrails: several middleware stacked together.
- ◆ **Common uses** → preventing PII leaks, blocking prompt injection, blocking dangerous requests, requiring approval for financial operations, and enforcing response safety standards.

Compare:

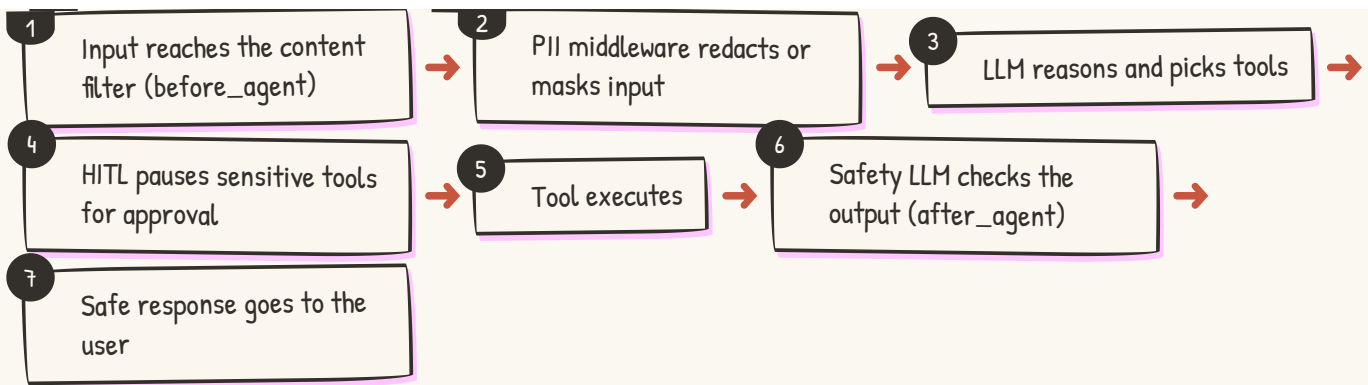
	<i>Deterministic</i>	<i>Model-based</i>
Mechanism	Regex / keyword lists	LLM classifier prompt
Cost	Zero LLM cost	Cost on every call
Semantics	Doesn't understand them	Understands meaning
Failure	False positives (e.g. "explain how malware spreads" gets blocked)	Expense and latency

 **Big idea:** Use the built-in `PIIMiddleware` and `HumanInTheLoopMiddleware`, write your own `AgentMiddleware` subclasses with `before_agent` / `after_agent` hooks, and stack them in `create_agent(middleware=[...])`.

Key concepts:

- ◆ **Deterministic demo** → `deterministic_guardrail(text) → bool` checks against banned words (hack, exploit, malware, bomb). It blocks "hack into a database" and also, wrongly, "explain how malware spreads".
- ◆ **Model-based demo** → `gpt-4o-mini` at temperature 0 with the prompt "Is the following input safe? Reply only safe/unsafe". It correctly allows the general question about how malware spreads.
- ◆ PII strategies:
 - ◆ **redact** : replaces the value with a placeholder such as `[REDACTED_EMAIL]`.
 - ◆ **mask** : replaces characters with `****` but keeps the last digits.
 - ◆ **hash** : replaces the value with a hash.
 - ◆ **block** : raises an exception.
- ◆ **Custom detectors** → pass `detector=<regex>` (e.g. a 32-character API key pattern).
- ◆ **Human-in-the-loop** → `interrupt_on={"send_email": True, "delete_records": True, "search_web": False}`, plus `checkpointer=InMemorySaver()` and a thread-ID config. Resume with `Command(resume={"decisions":[{"type": "approve"}]})` or with `"reject"` and a message (e.g. "too risky, needs DBA review").
- ◆ **Custom before-agent guardrail** → `ContentFilterMiddleware(AgentMiddleware)` takes `banned_keywords`. In `before_agent(state, runtime)` it checks the first human message (lowercased) and, on a match, returns a refusal message with `jump_to: "end"`. It is registered with `@hook_config(can_jump_to=["end"])`.
- ◆ **Custom after-agent guardrail** → `SafetyGuardrailMiddleware` uses a safety LLM to judge whether the final AI response is safe and replaces it if not. Good for compliance disclaimers (legal, medical, financial), quality checks, and catching sensitive info that slipped through.
- ◆ **Layered example** → content filter, then PII, then HITL, then the output safety model. A healthcare chatbot is left as an exercise.

Flow (layered):



Code essentials:

```
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware, HumanInTheLoopMiddleware, Agent
agent = create_agent(model, tools=[customer_lookup], middleware=[
    PIIMiddleware("email", strategy="redact", apply_to_input=True),
    PIIMiddleware("credit_card", strategy="mask", apply_to_input=True),
    PIIMiddleware("api_key", detector=r"sk-[a-zA-Z0-9]{32}", strategy="block", apply_to
```

⚠ watch out

- With `block`, wrap `invoke` in `try/except`. It raises errors like "Detected 1 instance of `api_key`".
- HITL does nothing without a `checkpoint` and a `thread_id` config.
- Auto-approve harmless tools (search) and gate destructive ones (email, delete, payments).
- Before-agent blocking saves LLM cost. Run the cheap deterministic filters first and the model-based checks last.



Big idea: Every chatbot, RAG app or agentic workflow needs evaluation to choose models and validate outputs against ground truth. LangChain plus LangSmith is used for tracing and eval reports.

Key concepts:

- ◆ **Four eval families** → AI/LLM-as-judge, gold-standard (ground truth) comparison, functional tests, and human evaluation.
- ◆ **Related practices** → dataset construction, regression testing, human annotation.
- ◆ **Model choice** → cost matters, but accuracy on the specific use case matters more.
- ◆ **Eval dataset** → (input, expected output) pairs to compare generated responses against.
- ◆ **LangSmith** → a cloud dashboard for eval reports and per-node execution traces.

Gotchas/tips: You need to know how to build ground-truth data before comparing LLMs. The part ends at this point.

Part summary —

- ◆ Vectorless RAG answers by having the LLM traverse a JSON tree of a structured document's sections and cite section and page. The tree is plain JSON and can be stored on a file system, S3 or MongoDB.
- ◆ **Choose between them by the documents** → traditional RAG suits millions of mixed docs, cheap fast factoid lookups and mature tooling; vectorless suits long structured docs (reports, contracts) that need cross-section reasoning and explainability. Production systems are moving to hybrid.
- ◆ Deep agents go beyond shallow and ReAct agents with four components: a planning to-do list, specialized parallel sub-agents, a detailed system prompt, and a shared file system as persistent memory.
- ◆ **deepagents.create_deep_agent(model, tools, system_prompt)**
(built on LangGraph) automatically attaches to-do, summarization and file middleware. Output comes back in **messages** and **files**.
- ◆ Guardrails can be deterministic (free but semantically blind) or model-based (understands meaning but costs money). LangChain provides PII middleware (redact/mask/hash/block), human-in-the-loop (needs a checkpoint; approve or reject via **Command(resume=...)**), custom **before_agent** / **after_agent** hooks, and layered stacks. An evaluation module (LLM-as-judge, gold standard, functional, human, with LangSmith) begins at the end.



Part 6 · 9:28 – 11:13

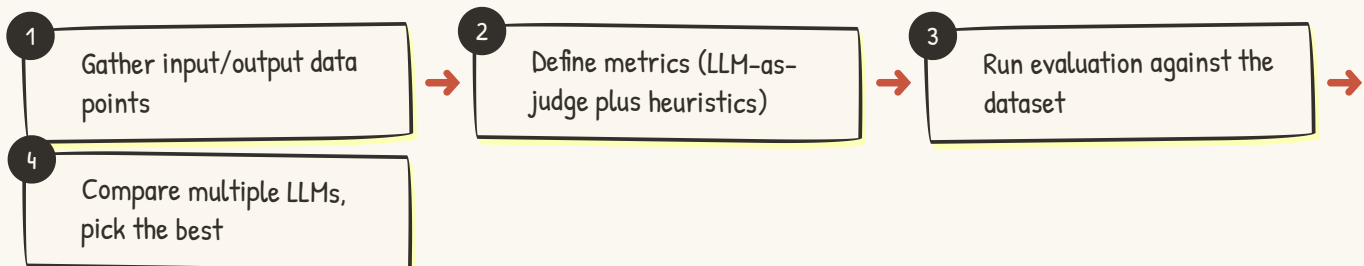
Evals + LLM Gateways

 **Big idea:** To evaluate an LLM app, you need ground-truth input/output pairs, a set of metrics (here scored by an LLM acting as judge), and experiments that compare candidate models. LangSmith stores the datasets and tracks the experiment results.

Key concepts

- ◆ **Data points** → input (question) paired with a reference output (ground truth).
- ◆ **LLM-as-a-judge** → a second LLM, driven by a grading prompt, scores the app's output.
- ◆ **LangSmith** → an observability and eval platform that works with or without LangChain. Its Evaluation area has **Datasets & Experiments**, Annotation Queues and Prompts. Deployment is handled by the LangGraph Platform.
- ◆ **Example** → one row of a dataset, holding `inputs` and `outputs` dicts.

Flow



Code essentials

- ◆ Install with `uv add langsmith openai`. The `.env` holds `LANGSMITH_API_KEY` (created in LangSmith under Settings → API Keys) and `OPENAI_API_KEY`. Set `LANGSMITH_TRACING=true`.

```
from langsmith import Client
client = Client()
ds = client.create_dataset(dataset_name="chatbot evaluation")
client.create_examples(dataset_id=ds.id, examples=[
    {"inputs": {"question": "What is LangChain?"},
    "outputs": {"answer": "A framework for building LLM applications"}}, ...])
```

 **watch out**

- Dataset names must be unique. Reusing one gives a "conflict" error.
- Annotation teams can produce the examples in CSV or Excel, and you can load them programmatically.
- In the UI, the "reference output" column is the ground truth.

Defining Chatbot Metrics: Correctness and Concision 9:42-9:50

 **Big idea:** An evaluator is just a Python function that takes `inputs`, `outputs` and `reference_outputs` and returns a bool or score. It can call an LLM judge or apply a plain rule.

Key concepts

- ♦ `wrappers.wrap_openai` : patches the OpenAI client so every call is traced in LangSmith. It supports the chat and responses APIs, both sync and async.
- ♦ **Correctness (LLM judge)** → the system prompt says "expert professor grading student answers". The user prompt holds the question, the real answer and the predicted answer, and asks the judge to reply "CORRECT" or "INCORRECT".
- ♦ **Concision (heuristic)** → passes if `len(output) < 2 * len(reference)`.

Code essentials

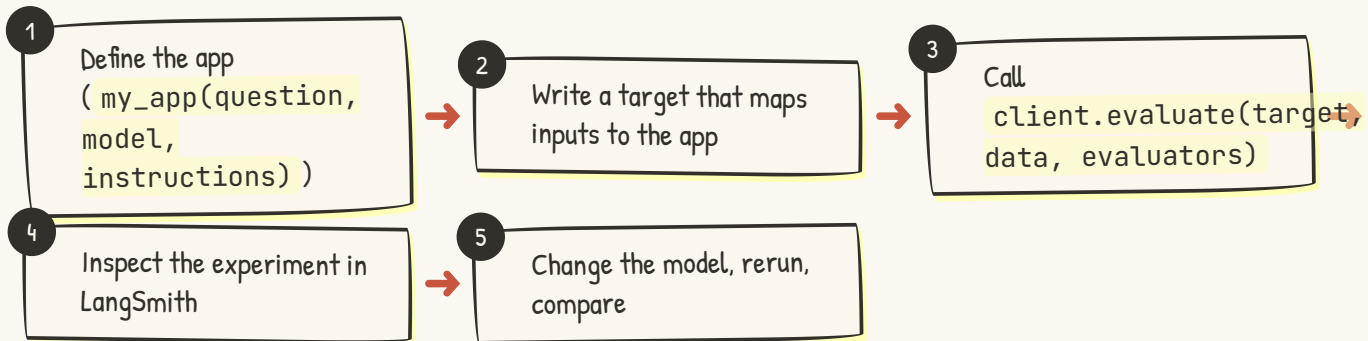
```
from langsmith import wrappers; import openai
oai = wrappers.wrap_openai(openai.OpenAI())
def correctness(inputs, outputs, reference_outputs) → bool:
    r = oai.chat.completions.create(model="gpt-4o-mini", temperature=0,
    messages=[{"role": "system", "content": eval_instructions},
              {"role": "user", "content": user_content}])
    return r.choices[0].message.content == "CORRECT"
```

watch out

- Use temperature 0 for the judge so its grades are deterministic.
- Turn the judge's text verdict into a boolean so it can be aggregated.

 **Big idea:** Wrap the chatbot in a target function, then call `client.evaluate` with the dataset and evaluators. Each run becomes a named experiment in LangSmith, and the experiments can be compared side by side.

Flow



Code essentials

```
def ls_target(inputs: dict) → dict:
    return {"response": my_app(inputs["question"], model="gpt-4o-mini")}
results = client.evaluate(ls_target, data="chatbot evaluation",
    evaluators=[correctness, concision], experiment_prefix="openai-4o-mini-chatbot")
```

Compare

Experiment	Correctness	Concision
gpt-4o-mini	~0.6	~0.4
gpt-4-turbo	~0.6	~0.2
Verdict	gpt-4o-mini wins (same accuracy, more concise, cheaper)	

watch out

- The default instruction ("respond in one short sentence") affects the concision scores.
- The experiment view shows input, reference, output and per-metric scores, plus latency and token cost.

→ Pairwise experiments are also available.

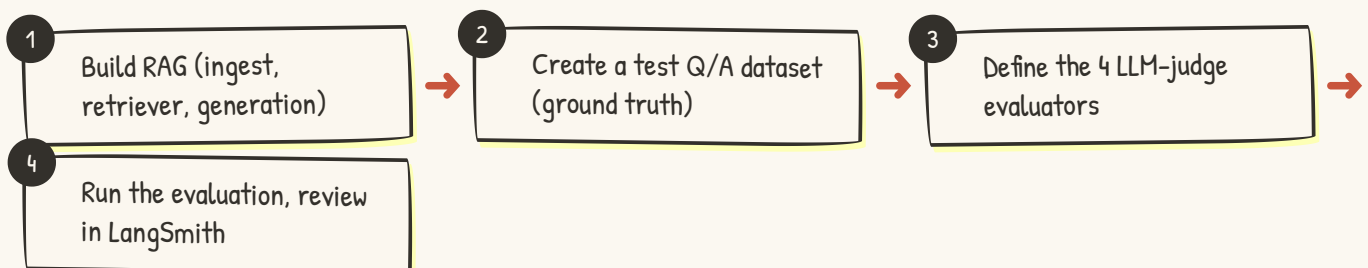
RAG Evaluation: The Four Metrics 9:58-10:02

 **Big idea:** A RAG pipeline can fail at retrieval or at generation, so it is scored on four axes, each judged by an LLM. The diagram comes from the LangSmith docs.

Key concepts / Compare

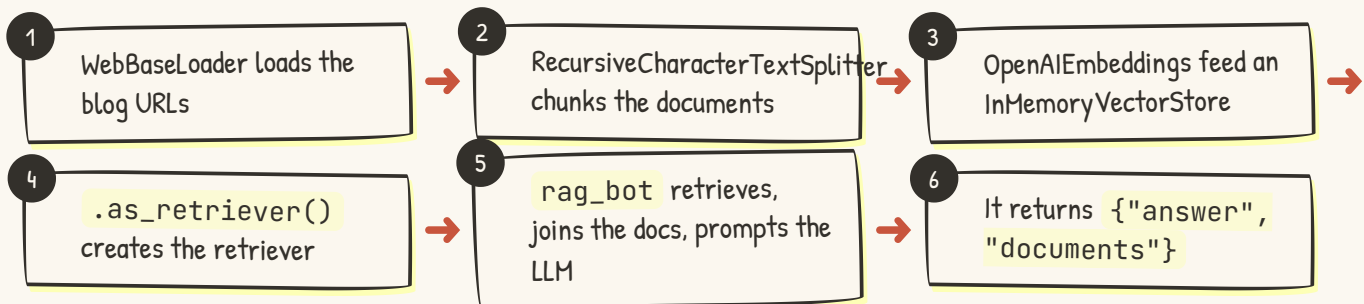
Metric	Compares	Question asked	Needs ground truth?
Correctness	response vs reference answer	Does it match the ground truth?	Yes
Answer relevance	response vs input	Does it address the question?	No
Groundedness	response vs retrieved docs	Is it supported by the docs (no hallucination)?	No
Retrieval relevance	retrieved docs vs input	Are the fetched docs relevant?	No

Flow



 **Big idea:** This is a standard in-memory RAG over three blog posts (agents, prompt engineering, adversarial attacks on LLMs). The bot returns both the answer and the retrieved documents, so the evaluators can inspect each.

Flow



Code essentials

```
from langsmith import traceable
from langchain.chat_models import init_chat_model
llm = init_chat_model("openai:gpt-4o-mini")
@traceable
def rag_bot(question: str) → dict:
    docs = retriever.invoke(question)
    ctx = "\n".join(d.page_content for d in docs)
    msg = llm.invoke([{"role": "system", "content": instr+ctx}, {"role": "user", "content":
    return {"answer": msg.content, "documents": docs}
```

- ◆ The prompt tells the model to act as a helpful assistant, use the retrieved context, and answer in at most 3 sentences.
- ◆ The test dataset is "rag test evaluation", with 3 Q/A pairs, for example "How does the ReAct agent use self-reflection?" plus its ground-truth answer. It is created with `create_dataset` and `create_examples`.

watch out

- `@traceable` traces any function in LangSmith automatically.
- The free LangSmith tier can return rate-limit warnings.



Big idea: Each metric gets the same three parts: a TypedDict output schema (explanation plus a boolean), a grading prompt, and a judge LLM bound to that schema. Only the inputs fed to the judge change from metric to metric.

Key concepts

- ◆ **Output schema** → explanation (reasoning) plus a bool such as correct, relevant or grounded.
- ◆ **Correctness prompt** → "teacher grading a quiz". The judge grades only factual accuracy against the ground truth, checks for conflicting statements, and accepts extra information if it is accurate. It explains step by step.
- ◆ **Relevance prompt** → is the answer concise, relevant, and does it help answer the question?
- ◆ **Groundedness prompt** → is the answer within the given facts, with nothing hallucinated?
- ◆ **Retrieval relevance prompt** → flag facts that are completely unrelated to the question.

Code essentials

```
from typing_extensions import Annotated, TypedDict
class CorrectnessGrade(TypedDict):
    explanation: Annotated[str, ..., "Explain your reasoning for the score"]
    correct: Annotated[bool, ..., "True if the answer is correct, False otherwise"]
grader = ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output(
    CorrectnessGrade, method="json_schema", strict=True)
def correctness(inputs, outputs, reference_outputs) → bool:
    return grader.invoke([sys_prompt, user_msg])["correct"]
```

- ◆ **To run** → client.evaluate(target, data="rag test evaluation", evaluators=[correctness, groundedness, relevance, retrieval_relevance], experiment_prefix=..., metadata={...}), then .to_pandas().

Inputs to each evaluator: correctness takes inputs["question"],

reference_outputs["answer"] and outputs["answer"]. Relevance takes the question and the answer. Groundedness takes outputs["documents"] and the answer. Retrieval relevance takes the question and the documents.

⚠ *watch out*

- An "invalid schema" error came from writing the `Annotated` fields wrong. Each needs a type, then `...`, then the description string (description last).
- `to_pandas()` needs pandas installed.
- **Results** → relevance 1.0, correctness 1.0, groundedness about 0.5. The UI also shows latency and token cost.
- You can add custom metrics if your pipeline has extra stages.

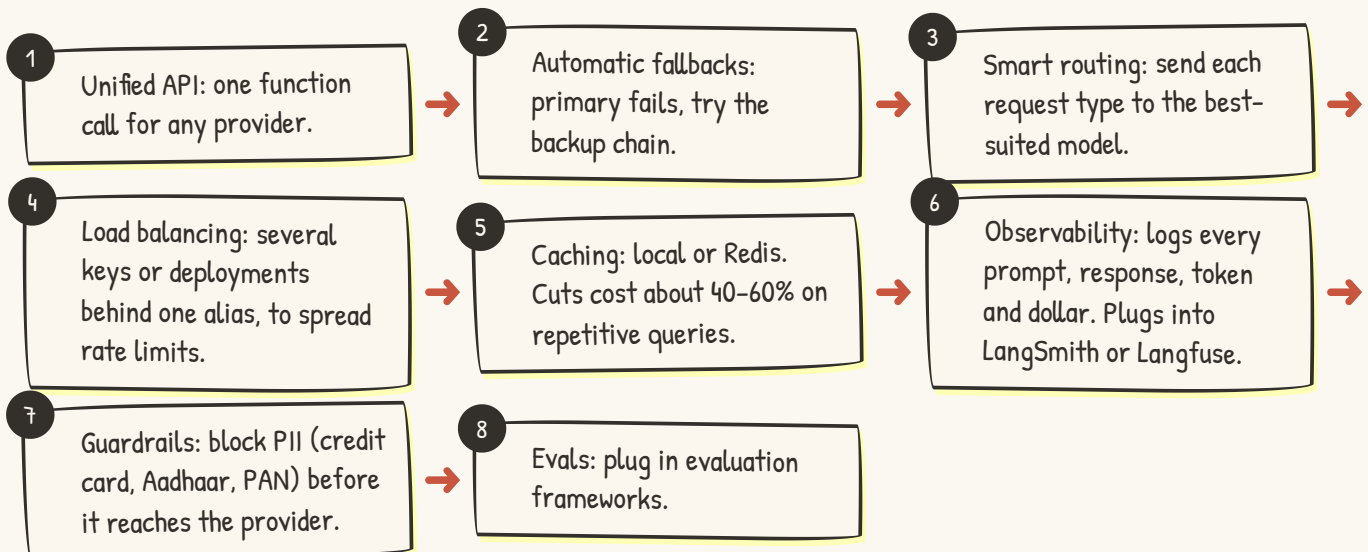
 **Big idea:** An LLM gateway is a smart middleware layer between your apps and the LLM providers. Apps call one API, and the gateway handles routing, fallbacks, caching, rate limiting, guardrails, cost tracking and evals. Switching models becomes a config change, not a code change.

Motivation: On 8 Nov 2023 OpenAI had an outage of about 4 hours. Apps hardcoded to OpenAI went down, including support bots at Cursor and Notion AI. With a gateway, traffic would have fallen back to another provider.

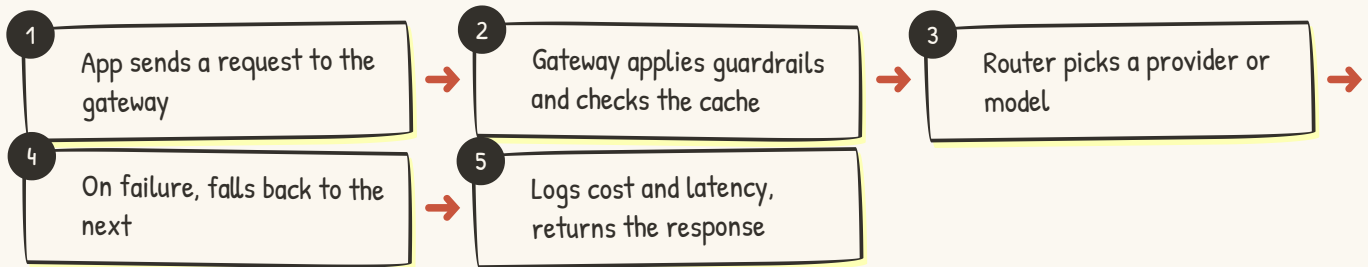
Compare

Without gateway	With gateway
A separate SDK or integration per provider	One unified API for 100+ providers
No fallback when a provider is down	Automatic fallbacks
No central cost tracking	Centralized logging, cost, rate limits
Switching models means rewriting code	Swap via config
You pay twice for repeated queries	Cache repeated queries

Key concepts (8 core capabilities)



Flow



Tool: LiteLLM (litellm.ai), an open-source gateway that also has an enterprise tier. It covers cost tracking, batches, guardrails, model access and budgets.



Big idea: `litellm.completion()` is one function for every provider. You select the provider through a prefix on the model string. Fallbacks, cost tracking and caching are just parameters or settings on that function.

Code essentials

- ◆ **Libraries** → `litellm`, `langchain`, `langchain-community`, `langchain-openai`, `python-dotenv`. Keys: OpenAI, Groq and Anthropic (Anthropic and Gemini were missing on purpose, to show failures).

```
from litellm import completion, completion_cost
r = completion(model="gpt-4o-mini", messages=[{"role": "user", "content": "Explain RAG in"}
r = completion(model="groq/llama-3.3-70b-versatile", messages=msgs) # same API
r = completion(model="gemini/gemini-1.5-flash", messages=msgs,
               fallbacks=["gpt-4o-mini", "groq/llama-3.3-70b-versatile"]) # primary
print(r.model, completion_cost(completion_response=r))
import litellm; from litellm.caching import Cache
litellm.cache = Cache(type="local"); completion(..., caching=True)
```

Key results

- ◆ **Fallback** → Gemini returned 403 (no key) and the response came from gpt-4o-mini. A fake OpenAI model name also fell back cleanly. The error is logged and execution continues.
- ◆ **Cost** → LiteLLM's built-in pricing database computes the cost of each call (the example showed 14 input tokens). Tag calls by team or project to see who is spending the budget.
- ◆ **Caching** → the first call took 1.45 s and the repeat took 0.0021 s, about 700x faster at zero cost.



watch out

- Reset global state before the caching demo: `litellm.callbacks`, `success_callback`, `failure_callback`, their `async` variants, and `litellm.cache = None`.
- `Cache(type="local")` is in-memory only. Use Redis for shared or production caching.



Big idea: `litellm.Router` maps abstract model aliases to concrete deployments. The app calls an alias and the router picks the deployment based on configuration and a routing strategy.

Key concepts

- ◆ **Task-to-model mapping** → coding goes to Claude Sonnet or GPT-4o, cheap summaries to GPT-4o-mini, very fast replies to Groq Llama, complex reasoning to Claude Opus.
- ◆ **model_list** : entries of `{"model_name": alias, "litellm_params": {"model": ..., "api_key": ...}}`.
- ◆ **Load balancing** → several entries share one alias (for example "gpt-pool"), and the router spreads calls across them.

Code essentials

```
from litellm import Router
router = Router(model_list=[
    {"model_name": "fast-cheap", "litellm_params": {"model": "groq/llama-3.3-70b-versatile",
    {"model_name": "smart-coding", "litellm_params": {"model": "gpt-4o", "api_key": "OAI"}}},
    routing_strategy="simple-shuffle")
router.completion(model="smart-coding", messages=[...])
```

Compare (routing strategies)

Strategy	Behavior
simple-shuffle	Spreads requests randomly across deployments; in the demo of 6 calls, they alternated between Groq and OpenAI
least-busy	Sends to the deployment with the fewest in-flight requests
latency-based-routing	Tracks recent response times and picks the fastest (Groq usually won)



watch out

→ The response metadata exposes the deployment ID and latency, so you can verify which backend served each call.

LangChain Integration via ChatLiteLLM 11:01-11:04

 **Big idea:** The `langchain-litellm` package provides `ChatLiteLLM`, a LangChain chat model backed by LiteLLM. It works in LCEL chains and with LangChain's own `.with_fallbacks`.

Code essentials

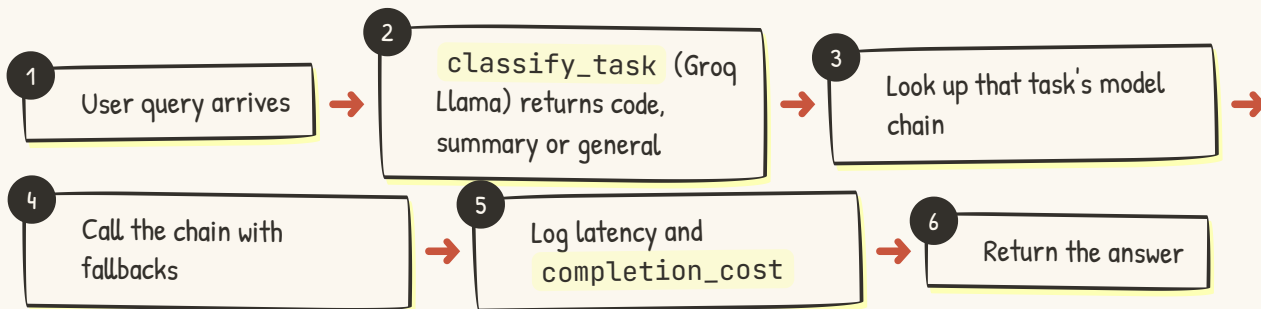
```
from langchain_litellm import ChatLiteLLM
primary = ChatLiteLLM(model="gpt-x") # non-existent model, will fail
fb1 = ChatLiteLLM(model="gpt-4o-mini", temperature=0.2)
fb2 = ChatLiteLLM(model="groq/llama-3.3-70b-versatile")
robust = primary.with_fallbacks([fb1, fb2])
chain = prompt | robust | StrOutputParser()
chain.invoke({"question": "Top 3 benefits of an LLM gateway?"})
```

- ◆ Uses `ChatPromptTemplate.from_messages` with a system message ("You are an AI engineer, always reply in JSON") and the user message. The primary model failed and the answer came from fb1 (gpt-4o-mini), with no SDK changes.



Big idea: A cheap, fast LLM classifies each query as code, summary or general. Each class has its own ordered model chain with fallbacks, and every call logs latency and cost.

Flow



Routing table

Task	Model chain (primary → fallbacks)
code	gpt-4o → gpt-4o-mini → groq/llama-3.3-70b
summary	gpt-4o-mini → groq/llama-3.3-70b
general (default)	groq/llama-3.3-70b → gpt-4o-mini

- ♦ **Results** → the Fibonacci function went to gpt-4o. "Summarize the attention mechanism" went to gpt-4o-mini (about 1.94 s). An elephant fun fact went to Llama 3.3 (fastest, near-zero cost on Groq's free tier).
- ♦ `routing.get(task, default_chain)` handles unknown labels.



Guardrails via LiteLLM Callbacks: PII Redaction and Prompt-Injection Blocking

11:08-11:13

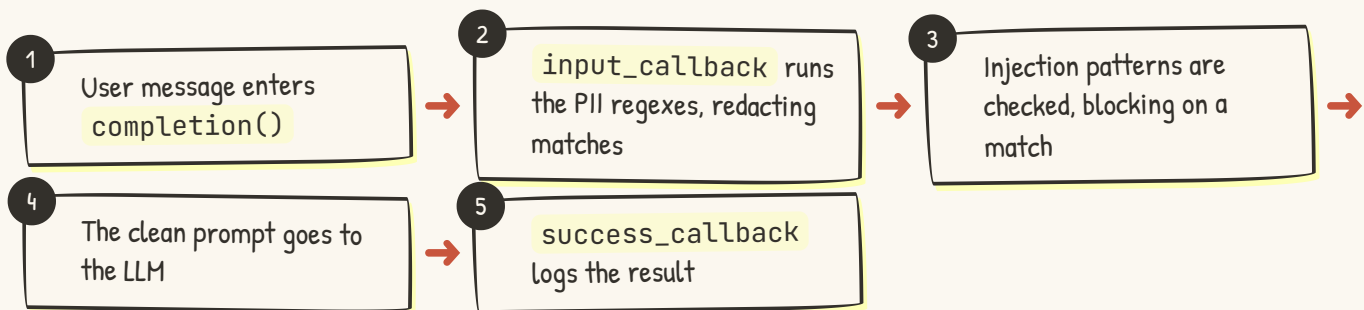


Big idea: LiteLLM exposes callback hooks. Guardrails belong in the pre-call (input) hook so sensitive or malicious content never reaches the LLM.

Key concepts

- ♦ **input_callback** : runs before the LLM call and can inspect or modify the prompt. This is where guardrails go.
- ♦ **success_callback** : runs after a successful call. Use it for logging and cost.
- ♦ **PII redaction** → regex patterns for email, Indian and US phone numbers, SSN, Aadhaar, PAN, credit card and IP address. Matches are replaced with placeholders such as `[EMAIL_REDACTED]`, and the detected types and counts are logged.
- ♦ **Prompt-injection blocking** → compiled regexes for phrases like "ignore all previous/prior/above instructions/prompts/rules", "reveal your prompt", and "you are now DAN with no restrictions". A match is flagged as "prompt injection detected" and blocked.

Flow



Code essentials

```
def redact_pii(text):
    for name, pat in PII_PATTERNS.items(): text = re.sub(pat, f"[{name}_REDACTED]", text)
    return text
litellm.input_callback = [pii_input_guardrail]
```



watch out

- In the demo the LLM never saw the real email, phone, PAN or Aadhaar. It even advised the user not to share personal data.
- A benign query ("capital of France?") passes through. Regex guardrails are simple but brittle, so keep extending the patterns. The speaker generated the injection patterns with ChatGPT.
- LiteLLM is one of several gateway libraries.

Part summary —

- ◆ **Evaluation pipeline** → build a ground-truth dataset in LangSmith (`create_dataset` and `create_examples`), write evaluator functions, run `client.evaluate` with an `experiment_prefix` , and compare experiments to choose a model. gpt-4o-mini beat gpt-4-turbo on concision at equal correctness.
- ◆ LLM-as-a-judge evaluators are a grading prompt, a temperature=0 judge, and a structured output (TypedDict with `Annotated[type, ..., "desc"]` , `method="json_schema"` , `strict=True`). Cheap heuristics such as length-based concision sit alongside them.
- ◆ **RAG is scored on four axes** → correctness (vs ground truth), answer relevance (vs question), groundedness (vs retrieved docs) and retrieval relevance (docs vs question). The RAG bot must return both the answer and the documents.
- ◆ An LLM gateway (LiteLLM) puts every provider behind one `completion()` call. Fallbacks, `completion_cost` tracking and caching (about 700x faster on cache hits) are configuration, not code changes.
- ◆ **litellm.Router** handles alias-based smart routing and load balancing (simple-shuffle, least-busy, latency-based). LangChain integrates via `ChatLiteLLM` plus `.with_fallbacks` . Pre-call `input_callback` hooks implement guardrails such as PII redaction and prompt-injection blocking.

❀ *you made it to the end* ❀